Hospital Management System

Keith Dmonte

October 25, 2026

CONTENTS

Hospital Management System

1. Problem Statement

    1.1 Title of the Project

    1.2 Background of the Problem

    1.3 Existing Problem or Limitation

    1.4 Proposed Database Solution

    1.5 Objectives

    1.6 Scope

2. Entity-Relationship (ER) Model

    2.1 Entities

    2.2 Attributes

    2.3 Relationships, Cardinality and Participation

    2.4 ER Diagram (Chen Notation)

    2.5 ER Diagram (Crow’s Foot Notation)

3. Relational Model (ER to Relational Mapping)

    3.1 Mapping Rules Applied

    3.2 Relational Schema (summary)

    3.3 Table: patients

    3.4 Table: doctors

    3.5 Table: appointments

    3.6 Keys Summary

    3.7 Data Type Choices

    3.8 Normalisation

4. System Architecture

    4.1 Three-Tier Architecture

    4.2 Technology Stack

4.3 Project Folder Structure

4.4 Request Flow Example: Adding a Patient

5. Database Implementation (SQL)

5.1 DDL: CREATE DATABASE

5.2 DDL: CREATE TABLE

5.3 DDL: ALTER TABLE

5.4 Constraints Used

5.5 DML: INSERT

5.6 DML: UPDATE

5.7 DML: DELETE

5.8 Query Operations

SELECT + ORDER BY

SELECT + WHERE (search, with ILIKE pattern matching)

JOIN (appointments with patient and doctor names)

Aggregate functions + scalar subqueries (Dashboard totals)

GROUP BY (Dashboard: appointments by status)

JOIN + WHERE + ORDER BY + LIMIT (Dashboard: upcoming appointments)

JOIN + GROUP BY + COUNT/SUM + HAVING (Report 1: Doctor workload and revenue)

Correlated subquery with NOT EXISTS (Report 2a: Patients without appointments)

Nested subquery with AVG (Report 2b: Patients older than average)

5.9 Query Coverage Checklist

6. Sample Data

6.1 Patients (10 records, master table)

6.2 Doctors (6 records, master table)

6.3 Appointments (15 records, transaction table)

7. Frontend Prototype

7.1 Home / Dashboard

7.2 Add / Insert Record

7.3 Update Record

7.4 Delete Record

7.5 Search Record

7.6 Reports

7.7 Screens vs Guideline

8. Backend: REST API

8.1 Endpoints

8.2 Response Codes

8.3 Example

9. Code Walkthrough

9.1 db.js: database connection

9.2 server.js: application entry point

9.3 routes/patients.js, doctors.js, appointments.js: CRUD

9.4 routes/reports.js: dashboard and reports

9.5 public/app.js: frontend logic

9.6 public/index.html and style.css

9.7 Security Measures

10. Installation and Running

10.1 Prerequisites

10.2 Steps

10.3 Viewing Data in pgAdmin

11. Testing

12. Guideline Compliance Summary

13. Limitations and Future Scope

14. Conclusion

Appendix: Complete Source Code

schema.sql

db.js

server.js

package.json

.env.example

routes/patients.js

routes/doctors.js

routes/appointments.js

routes/reports.js

public/index.html

public/app.js

public/style.css

Hospital Management System

DBMS Micro Project Report

A working database application for managing patients, doctors and appointments, built with PostgreSQL, Node.js/Express and a plain HTML/CSS/JavaScript frontend.

Project	Hospital Management System (HMS)
Database	PostgreSQL 18
Backend	Node.js 20 + Express 4 + node-postgres (`pg`)
Frontend	HTML5, CSS3, vanilla JavaScript (`fetch()` API)
Author	Keith Dmonte
Date	October 2026

1. Problem Statement

1.1 Title of the Project

Hospital Management System: a relational database for patient, doctor and appointment records.

1.2 Background of the Problem

Small and mid-sized clinics and hospitals handle three core kinds of information every day:

- **Patients**: who they are, how to contact them, their age and gender.
- **Doctors**: their names, specialties, contact numbers and consultation fees.
- **Appointments**: which patient sees which doctor, on what date, for what reason, and whether the visit is scheduled, completed or cancelled.

In many smaller clinics this information still lives in paper registers, appointment diaries, or loose spreadsheets kept by the front desk. Each doctor may have their own diary, and patient details are re-written every time a patient visits.

1.3 Existing Problem or Limitation

Paper registers and unconnected spreadsheets cause several practical problems:

1. **Duplicate and inconsistent data.** The same patient gets written down many times with slightly different spellings or phone numbers. There is no single source of truth.
2. **No integrity checks.** Nothing stops a receptionist from booking an appointment for a doctor who has left, entering an age of -5, or writing a status like “done?” instead of a fixed value.
3. **Slow searching.** Finding all visits of one patient, or all appointments of one doctor, means flipping through pages or scrolling through files.
4. **No reporting.** Management questions such as “which doctor is the busiest?”, “how much consultation revenue did each doctor bring in?” or “which registered patients have never booked a visit?” cannot be answered without manual counting.
5. **Orphaned records.** When a patient record is removed, their appointment entries are left behind and point to nobody.
6. **Double booking.** The same patient can accidentally be booked twice with the same doctor on the same day.

1.4 Proposed Database Solution

We propose a **centralised relational database** (PostgreSQL) with a **web-based frontend** that the front-desk staff can use from any browser. The solution:

- Stores patients, doctors and appointments in **three normalised tables** linked by **foreign keys**, so every fact is stored once.
- Enforces **data integrity inside the database** using `PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`, `UNIQUE`, `CHECK` and `DEFAULT` constraints. Bad data is rejected even if the frontend has a bug.
- Uses `ON DELETE CASCADE` so that deleting a patient or doctor automatically removes their appointments, leaving no orphan rows.
- Blocks double booking with a composite `UNIQUE (patient_id, doctor_id, appt_date)` constraint.
- Provides **Add, Update, Delete and Search** screens for every table.
- Provides a **Dashboard** and **Reports** built with SQL aggregate functions, `GROUP BY`, `HAVING`, joins and subqueries, answering the management questions listed above instantly.

1.5 Objectives

1. Design an ER model for a hospital's core workflow.
2. Map the ER model to a relational schema in at least Third Normal Form.
3. Implement the schema in PostgreSQL with full DDL, DML and query operations.
4. Load realistic sample data.
5. Build a frontend that performs all CRUD operations, search and reporting against the live database.

1.6 Scope

In scope: patient registration, doctor registration with consultation fee, appointment booking and status tracking, search, dashboard statistics, workload/revenue and patient reports.

Out of scope (future work): user login and roles, billing/invoices, prescriptions, lab results, time slots within a day.

2. Entity-Relationship (ER) Model

2.1 Entities

Entity	Type	Description
Patient	Strong entity	A person registered with the hospital.
Doctor	Strong entity	A medical practitioner who sees patients.
Appointment	Strong entity (associative)	One booked visit of one patient with one doctor on one date. It resolves the many-to-many relationship between Patient and Doctor and carries its own attributes (date, reason, status).

2.2 Attributes

Patient

Attribute	Kind	Notes
id	Simple, single-valued	**Primary key** (surrogate, auto-generated)
name	Simple	Required
age	Simple	Optional, 0 to 150
gender	Simple	Male / Female / Other
phone	Simple	Unique (candidate key)
address	Simple	Optional free text

Doctor

Attribute	Kind	Notes
id	Simple	**Primary key**
name	Simple	Required
specialty	Simple	e.g. Cardiology
phone	Simple	Unique (candidate key)
consultation_fee	Simple	Non-negative amount in rupees, default 500

Appointment

Attribute	Kind	Notes
id	Simple	**Primary key**
appt_date	Simple	Required
reason	Simple	Optional
status	Simple	scheduled / completed / cancelled, default scheduled

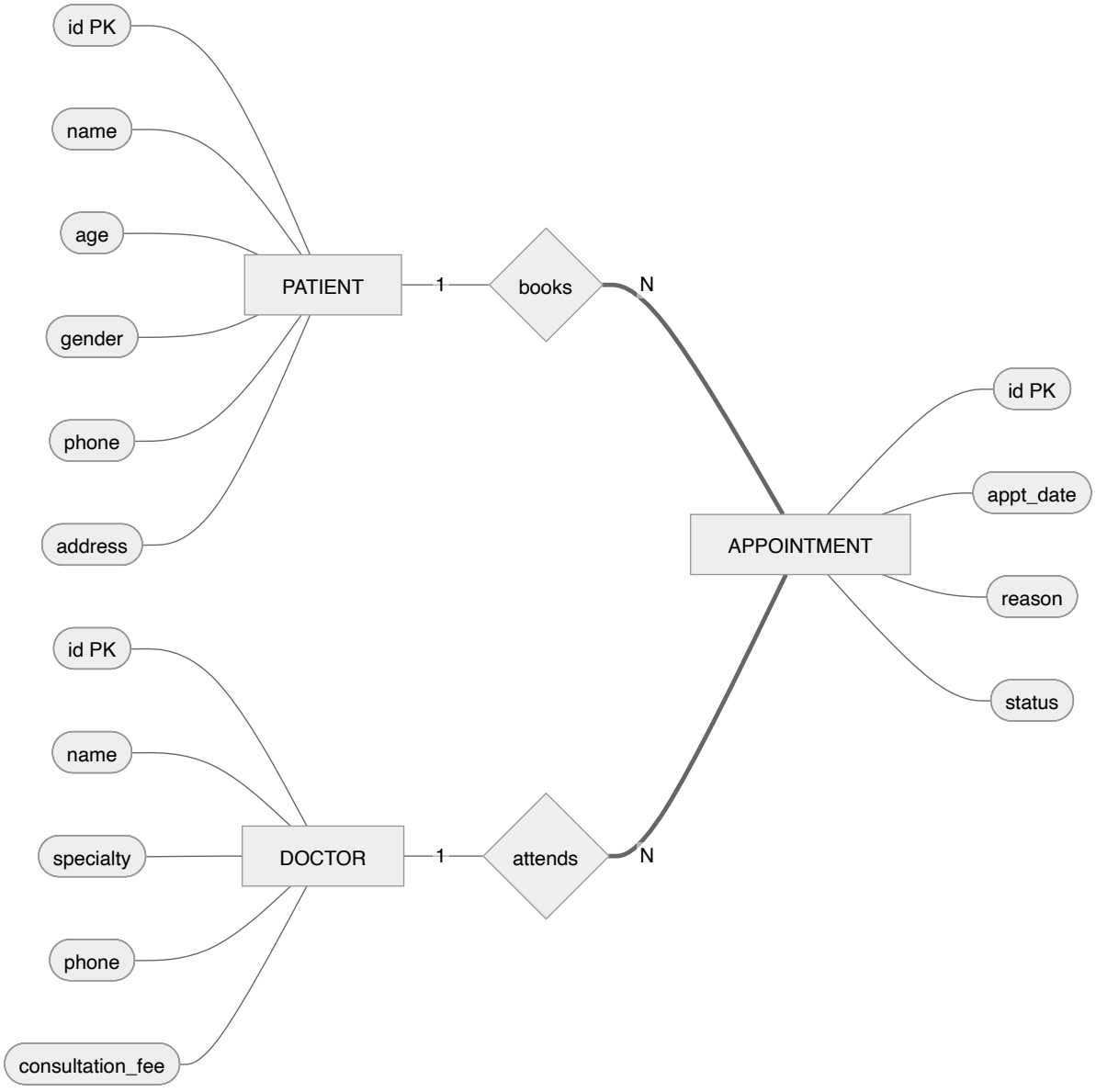
2.3 Relationships, Cardinality and Participation

Relationship	Between	Cardinality	Participation
books	Patient - Appointment	**1 : N** (one patient books many appointments; each appointment belongs to exactly one patient)	Patient: **partial** (a patient may exist with zero appointments). Appointment: **total** (every appointment must have a patient).
attends	Doctor - Appointment	**1 : N** (one doctor attends many appointments; each appointment is with exactly one doctor)	Doctor: **partial** (a new doctor may have zero appointments). Appointment: **total** (every appointment must have a doctor).

Taken together, **Patient and Doctor have an M : N relationship** (“a patient can consult many doctors, and a doctor sees many patients”), which is resolved through the Appointment entity.

2.4 ER Diagram (Chen Notation)

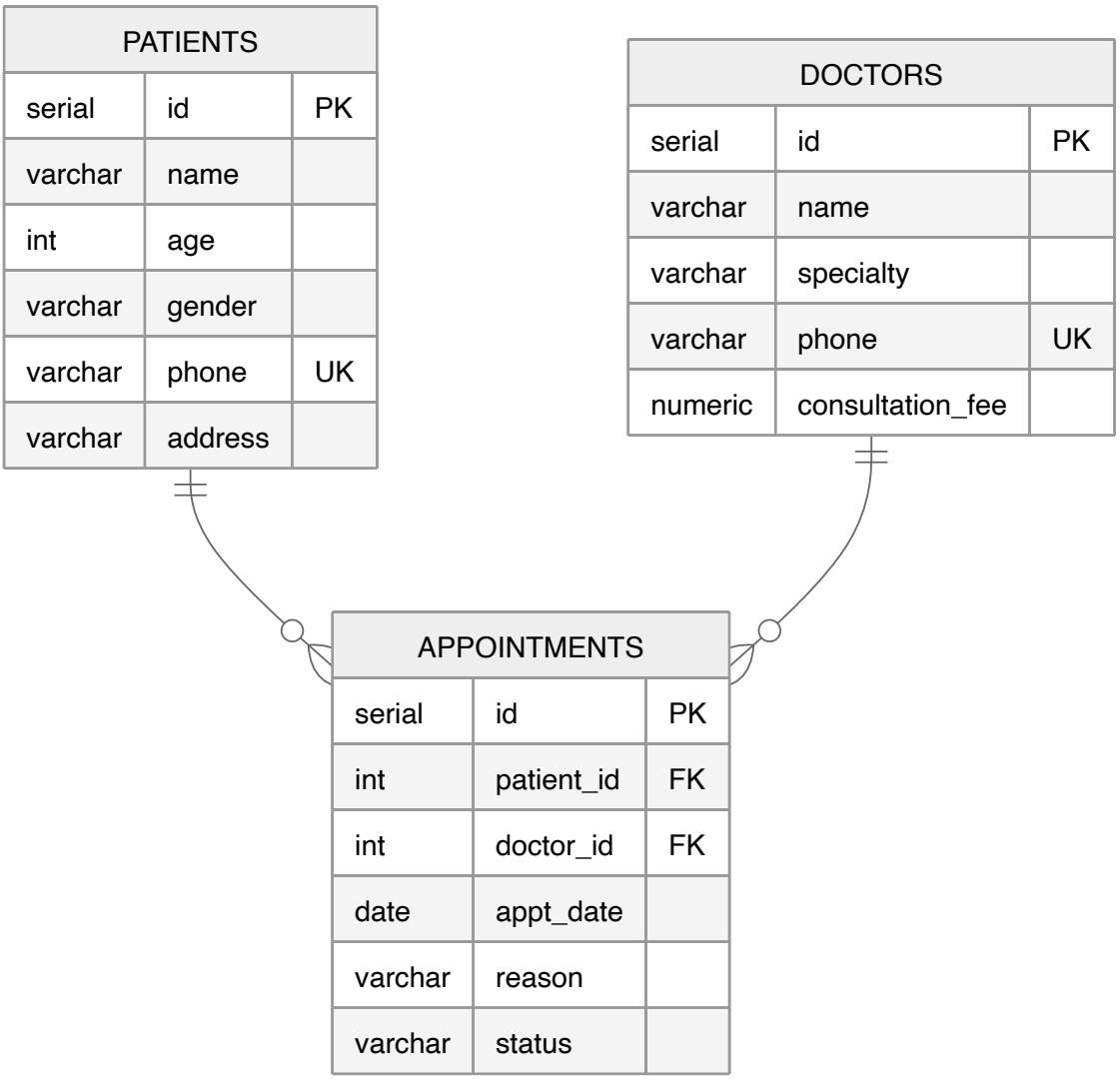
Rectangles are entities, ovals are attributes (PK marked), diamonds are relationships. Thick lines show **total participation**, thin lines show **partial participation**. Edge labels show cardinality.



ER diagram, Chen notation

2.5 ER Diagram (Crow's Foot Notation)

The same model in crow's-foot notation (PATIENTS books APPOINTMENTS, DOCTORS attends APPOINTMENTS). `||` means "exactly one" (mandatory), `o{` means "zero or many" (optional many).



ER diagram, crow's foot notation

3. Relational Model (ER to Relational Mapping)

3.1 Mapping Rules Applied

1. **Each strong entity becomes a table.** Patient becomes `patients`, Doctor becomes `doctors`, Appointment becomes `appointments`. Each simple attribute becomes a column.
2. **The entity key becomes the primary key.** Each table uses a surrogate key `id SERIAL PRIMARY KEY`.
3. **1 : N relationships** are mapped by putting the primary key of the "1" side into the "N" side as a **foreign key**. So `appointments.patient_id` references `patients(id)` and `appointments.doctor_id` references `doctors(id)`.
4. **Total participation** of Appointment in both relationships is enforced with `NOT NULL` on both foreign keys.
5. **The M : N** between Patient and Doctor needs no extra table, because Appointment already acts as the associative (junction) table.

3.2 Relational Schema (summary)

- **patients** (<u>id</u>, name, age, gender, phone, address)
- **doctors** (<u>id</u>, name, specialty, phone, consultation_fee)
- **appointments** (<u>id</u>, *patient_id* → patients.id, *doctor_id* → doctors.id, appt_date, reason, status)

Underlined = primary key, italic = foreign key.

3.3 Table: `patients`

Column	Data Type	Key	Constraints
id	`SERIAL` (auto-increment INT)	**PK**	`PRIMARY KEY`
name	`VARCHAR(100)`		`NOT NULL`
age	`INT`		`CHECK (age BETWEEN 0 AND 150)`
gender	`VARCHAR(10)`		`CHECK (gender IN ('Male','Female','Other'))`
phone	`VARCHAR(15)`	**Candidate / Alternate key**	`UNIQUE`
address	`VARCHAR(255)`		

3.4 Table: `doctors`

Column	Data Type	Key	Constraints
id	`SERIAL`	**PK**	`PRIMARY KEY`
name	`VARCHAR(100)`		`NOT NULL`
specialty	`VARCHAR(50)`		
phone	`VARCHAR(15)`	**Candidate / Alternate key**	`UNIQUE`
consultation_fee	`NUMERIC(8,2)`		`NOT NULL DEFAULT 500`, `CHECK (consultation_fee >= 0)` (added with `ALTER TABLE`)

3.5 Table: `appointments`

Column	Data Type	Key	Constraints
id	`SERIAL`	**PK**	`PRIMARY KEY`
patient_id	`INT`	**FK** → patients(id)	`NOT NULL`, `ON DELETE CASCADE`
doctor_id	`INT`	**FK** → doctors(id)	`NOT NULL`, `ON DELETE CASCADE`
appt_date	`DATE`		`NOT NULL`
reason	`VARCHAR(255)`		
status	`VARCHAR(20)`		`NOT NULL DEFAULT 'scheduled'`, `CHECK (status IN ('scheduled','completed','cancelled'))`
(patient_id, doctor_id, appt_date)		**Composite candidate key**	`UNIQUE` (constraint `uq_patient_doctor_date`, added with `ALTER TABLE`)

3.6 Keys Summary

Table	Primary Key	Foreign Keys	Candidate / Alternate Keys
patients	id	none	phone
doctors	id	none	phone
appointments	id	patient_id, doctor_id	(patient_id, doctor_id, appt_date)

Why surrogate keys? Names are not unique, and phone numbers can change or be missing. An auto-generated integer `id` never changes, so foreign keys pointing to it stay valid. Phone is still kept `UNIQUE` as an alternate key so two records cannot share a number.

3.7 Data Type Choices

Type	Used for	Reason
`SERIAL`	all `id` columns	Auto-incrementing integer, PostgreSQL creates a sequence automatically.
`VARCHAR(n)`	names, phone, address, status	Bounded text; length limits catch garbage input.
`INT`	age, foreign keys	Whole numbers; FKs must match the `SERIAL` (INT) type of the PK.
`NUMERIC(8,2)`	consultation_fee	Exact decimal for money. Never use floating point for currency.
`DATE`	appt_date	Calendar date without time; allows date comparison with `CURRENT_DATE`.

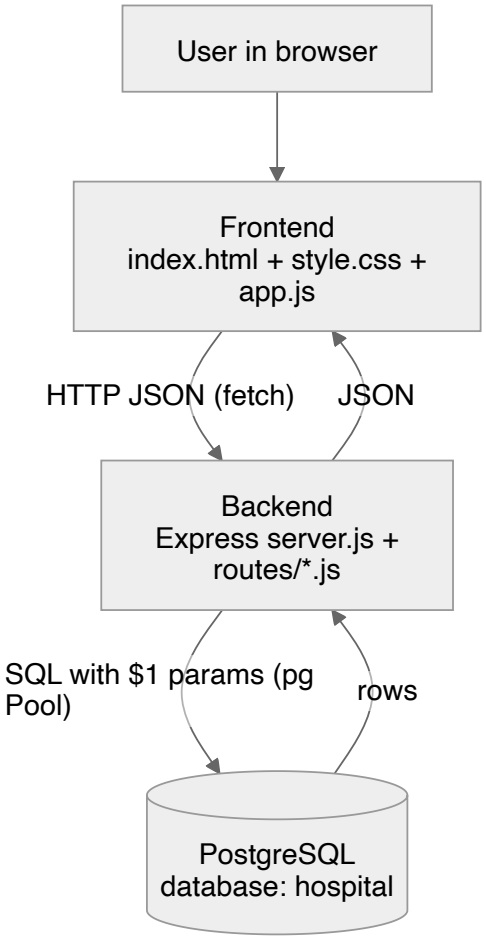
3.8 Normalisation

- **1NF:** every column holds a single atomic value (one phone, one address, one status). No repeating groups such as “doctor1, doctor2” columns.
- **2NF:** every table has a single-column primary key, so no partial dependency on part of a key is possible.
- **3NF:** no non-key attribute depends on another non-key attribute. For example, the doctor’s fee and specialty are stored only in `doctors`, not copied into `appointments`; the patient’s name is stored only in `patients`. Appointments refer to them by ID and the name is fetched with a `JOIN` when needed.

The schema is therefore in **Third Normal Form (3NF)**, and also in BCNF since every determinant is a candidate key.

4. System Architecture

4.1 Three-Tier Architecture



Three-tier architecture

Tier	Technology	Responsibility
Presentation	HTML, CSS, JavaScript	Forms, tables, tabs, search boxes, dashboard cards. Sends requests with `fetch()`.
Application	Node.js + Express	REST API. Validates input, runs SQL through the `pg` connection pool, returns JSON. Also serves the static frontend files.
Data	PostgreSQL 18	Stores data, enforces all constraints, executes queries.

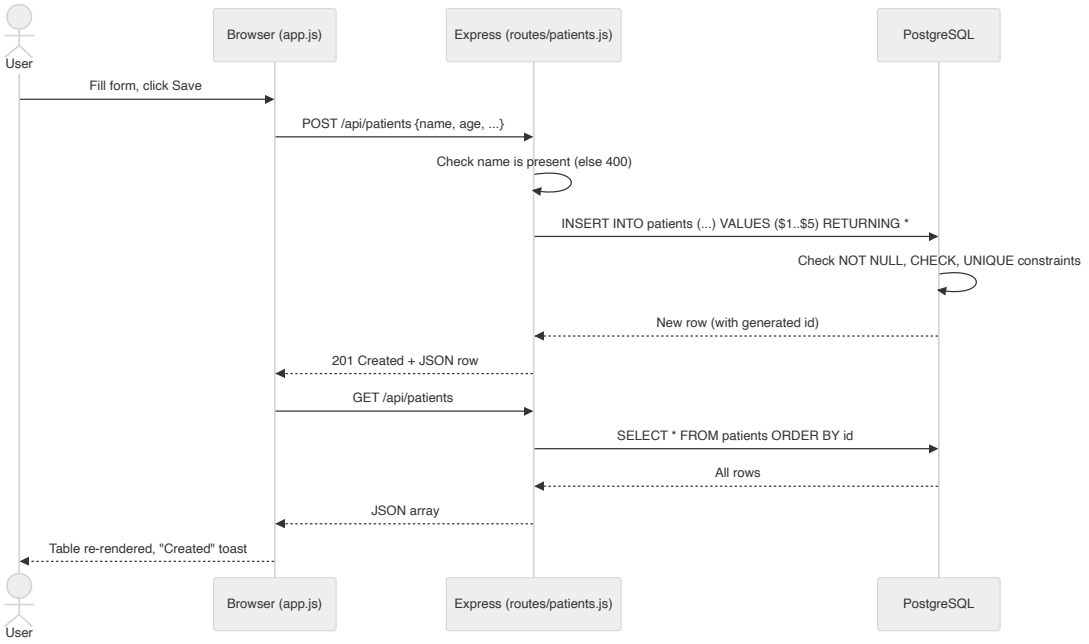
4.2 Technology Stack

Component	Version	Purpose
PostgreSQL	18 (Homebrew)	Relational DBMS
Node.js	20+	JavaScript runtime for the server
Express	4.21	Web framework: routing, JSON parsing, static files
pg (node-postgres)	8.13	PostgreSQL driver with connection pooling and parameterised queries
dotenv	16.4	Loads database URL and port from `.env`
pgAdmin	optional	GUI to browse tables while the app runs

4.3 Project Folder Structure

```
dbms-crud/
├── server.js           Express app: mounts API routes, serves /public
├── db.js               PostgreSQL connection pool + DATE type parser
├── schema.sql          DDL (database, tables, constraints) + seed data
├── package.json        Dependencies and npm scripts (start, dev, initdb)
├── .env / .env.example DATABASE_URL and PORT
├── routes/
│   ├── patients.js     CRUD + search for patients
│   ├── doctors.js      CRUD + search for doctors
│   ├── appointments.js CRUD + search for appointments (JOIN for names)
│   └── reports.js      Dashboard summary + 2 reports
├── public/
│   ├── index.html      All screens (tabs): Dashboard, Patients, Doctors,
│   │                   Appointments, Reports
│   ├── style.css       Styling
│   └── app.js          Frontend logic: fetch calls, rendering, search
└── docs/
    ├── Project_Report.md / .pdf   This document
    └── screenshots/               Screens used in Section 7
```

4.4 Request Flow Example: Adding a Patient



Create patient sequence

If a constraint fails (for example a duplicate phone number), PostgreSQL rejects the `INSERT`, the API returns the error message as JSON, and the UI shows it in a red toast.

5. Database Implementation (SQL)

All SQL lives in `schema.sql` (DDL + seed DML) and in the route files (DML + queries run by the application). The DBMS used is **PostgreSQL 18**.

5.1 DDL: CREATE DATABASE

PostgreSQL has no `CREATE DATABASE IF NOT EXISTS`, so the script builds the statement only when the database is missing and executes it with psql's `\gexec`. It then connects to the new database with `\c`.

```
SELECT 'CREATE DATABASE hospital'
 WHERE NOT EXISTS (SELECT FROM pg_database WHERE datname = 'hospital')\gexec

\c hospital
```

Tables are dropped in foreign-key-safe order (child first) so the script can be re-run:

```
DROP TABLE IF EXISTS appointments;
DROP TABLE IF EXISTS patients;
DROP TABLE IF EXISTS doctors;
```

5.2 DDL: CREATE TABLE

```
CREATE TABLE patients (
  id      SERIAL       PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  age     INT          CHECK (age BETWEEN 0 AND 150),
  gender  VARCHAR(10)  CHECK (gender IN ('Male', 'Female', 'Other')),
  phone   VARCHAR(15)  UNIQUE,                -- candidate key
  address VARCHAR(255)
);

CREATE TABLE doctors (
  id        SERIAL       PRIMARY KEY,
  name      VARCHAR(100) NOT NULL,
  specialty VARCHAR(50),
  phone     VARCHAR(15)  UNIQUE               -- candidate key
);

CREATE TABLE appointments (
  id         SERIAL      PRIMARY KEY,
  patient_id INT         NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
  doctor_id  INT         NOT NULL REFERENCES doctors(id)  ON DELETE CASCADE,
  appt_date  DATE        NOT NULL,
  reason     VARCHAR(255),
  status     VARCHAR(20) NOT NULL DEFAULT 'scheduled'
                         CHECK (status IN ('scheduled', 'completed',
'cancelled'))
);
```

5.3 DDL: ALTER TABLE

Two `ALTER TABLE` statements extend the schema after creation:

```
-- Add a new column with default + check constraint
ALTER TABLE doctors
  ADD COLUMN consultation_fee NUMERIC(8, 2) NOT NULL DEFAULT 500
             CHECK (consultation_fee >= 0);

-- Add a composite UNIQUE constraint: no double booking
ALTER TABLE appointments
  ADD CONSTRAINT uq_patient_doctor_date UNIQUE (patient_id, doctor_id,
appt_date);
```

5.4 Constraints Used

Constraint	Where	What it prevents
`PRIMARY KEY`	id on all tables	Duplicate or NULL identifiers
`FOREIGN KEY ... REFERENCES`	appointments.patient_id, doctor_id	Appointments pointing to non-existent patients/doctors
`ON DELETE CASCADE`	both FKs	Orphan appointments after a patient/doctor is deleted
`NOT NULL`	names, FKs, appt_date, status, fee	Missing mandatory data; enforces total participation
`UNIQUE`	patients.phone, doctors.phone	Two people sharing one phone number
`UNIQUE` (composite)	(patient_id, doctor_id, appt_date)	Double booking
`CHECK`	age, gender, status, consultation_fee	Out-of-range or invalid values
`DEFAULT`	status = 'scheduled', fee = 500	Sensible values when none is given

Constraint enforcement was tested against the live database:

Test	Result
Insert patient with an existing phone `9876500001`	`duplicate key value violates unique constraint "patients_phone_key"`
Insert patient with age `-3`	`new row for relation "patients" violates check constraint "patients_age_check"`
Create appointment without a date	Rejected: `patient_id, doctor_id and appt_date are required`
Delete a patient who has appointments	Their appointments are removed automatically (cascade)

5.5 DML: INSERT

Seed data (multi-row insert):

```
INSERT INTO patients (name, age, gender, phone, address) VALUES
  ('Anita Rao',      34, 'Female', '9876500001', '12 MG Road, Bengaluru'),
  ('Rahul Mehta',    52, 'Male',   '9876500002', '44 Park Street, Kolkata'),
  ...;
```

Insert from the application (`routes/patients.js`). Values are passed as parameters `$1..$5` , never pasted into the SQL string, which prevents SQL injection. `RETURNING *` sends back the new row including its generated `id` .

```
INSERT INTO patients (name, age, gender, phone, address)
VALUES ($1, $2, $3, $4, $5) RETURNING *;
```

Doctor insert uses `COALESCE` to fall back to the default fee when none is given:

```
INSERT INTO doctors (name, specialty, phone, consultation_fee)
VALUES ($1, $2, $3, COALESCE($4, 500)) RETURNING *;
```

5.6 DML: UPDATE

```
UPDATE patients
   SET name = $1, age = $2, gender = $3, phone = $4, address = $5
 WHERE id = $6 RETURNING *;

UPDATE doctors
   SET name = $1, specialty = $2, phone = $3,
       consultation_fee = COALESCE($4, consultation_fee)   -- keep old fee if
blank
 WHERE id = $5 RETURNING *;

UPDATE appointments
   SET patient_id = $1, doctor_id = $2, appt_date = $3, reason = $4, status =
$5
 WHERE id = $6 RETURNING *;
```

If no row matches the `WHERE id` , the API returns **404 Not Found**.

5.7 DML: DELETE

```
DELETE FROM patients     WHERE id = $1;
DELETE FROM doctors      WHERE id = $1;
DELETE FROM appointments WHERE id = $1;
```

Because of `ON DELETE CASCADE`, deleting a patient or doctor also deletes all of their appointments in the same statement.

5.8 Query Operations

Every query type required by the guidelines is used by a real screen in the application.

SELECT + ORDER BY

```
SELECT * FROM patients ORDER BY id;
```

SELECT + WHERE (SEARCH, WITH ILIKE PATTERN MATCHING)

Used by the search boxes. `$1::text IS NULL` makes the filter optional: with no search term every row is returned.

```
SELECT * FROM patients
 WHERE $1::text IS NULL
    OR name    ILIKE '%' || $1 || '%'
    OR phone   ILIKE '%' || $1 || '%'
    OR address ILIKE '%' || $1 || '%'
 ORDER BY id;
```

JOIN (APPOINTMENTS WITH PATIENT AND DOCTOR NAMES)

```
SELECT a.id, a.patient_id, a.doctor_id, a.appt_date, a.reason, a.status,
       p.name AS patient_name,
       d.name AS doctor_name
  FROM appointments a
  LEFT JOIN patients p ON p.id = a.patient_id
  LEFT JOIN doctors  d ON d.id = a.doctor_id
 WHERE $1::text IS NULL
    OR p.name ILIKE '%' || $1 || '%'
    OR d.name ILIKE '%' || $1 || '%'
    OR a.reason ILIKE '%' || $1 || '%'
    OR a.status ILIKE '%' || $1 || '%'
 ORDER BY a.id;
```

AGGREGATE FUNCTIONS + SCALAR SUBQUERIES (DASHBOARD TOTALS)

```
SELECT (SELECT COUNT(*) FROM patients)::int     AS patients,
       (SELECT COUNT(*) FROM doctors)::int      AS doctors,
       (SELECT COUNT(*) FROM appointments)::int AS appointments,
       (SELECT COUNT(*) FROM appointments
         WHERE status = 'scheduled' AND appt_date >= CURRENT_DATE)::int AS
upcoming;
```

GROUP BY (DASHBOARD: APPOINTMENTS BY STATUS)

```
SELECT status, COUNT(*)::int AS count
  FROM appointments
 GROUP BY status
 ORDER BY status;
```

status	count
cancelled	2
completed	8
scheduled	5

JOIN + WHERE + ORDER BY + LIMIT (DASHBOARD: UPCOMING APPOINTMENTS)

```
SELECT a.id, a.appt_date, a.reason, p.name AS patient_name, d.name AS
doctor_name
  FROM appointments a
  JOIN patients p ON p.id = a.patient_id
  JOIN doctors  d ON d.id = a.doctor_id
 WHERE a.status = 'scheduled' AND a.appt_date >= CURRENT_DATE
 ORDER BY a.appt_date
 LIMIT 5;
```

JOIN + GROUP BY + COUNT/SUM + HAVING (REPORT 1: DOCTOR WORKLOAD AND REVENUE)

`LEFT JOIN` keeps doctors with zero appointments. `FILTER (WHERE ...)` counts each status separately. `HAVING` filters **groups** after aggregation (unlike `WHERE`, which filters rows before). The minimum count `$1` comes from the “Min. appointments” input on the Reports screen.

```
SELECT d.id, d.name, d.specialty, d.consultation_fee,
       COUNT(a.id)::int                                          AS total,
       COUNT(a.id) FILTER (WHERE a.status = 'completed')::int AS completed,
       COUNT(a.id) FILTER (WHERE a.status = 'scheduled')::int AS scheduled,
       COUNT(a.id) FILTER (WHERE a.status = 'cancelled')::int AS cancelled,
       COALESCE(SUM(d.consultation_fee)
                FILTER (WHERE a.status = 'completed'), 0)     AS revenue
  FROM doctors d
  LEFT JOIN appointments a ON a.doctor_id = d.id
 GROUP BY d.id
HAVING COUNT(a.id) >= $1
 ORDER BY revenue DESC, total DESC;
```

Result with `$1 = 0` (all doctors):

Doctor	Specialty	Fee	Total	Completed	Scheduled	Cancelled	Revenue
Dr. Priya Nair	Cardiology	1200	4	3	1	0	3600
Dr. Meera Iyer	Dermatology	700	2	2	0	0	1400
Dr. Arjun Singh	Orthopedics	900	3	1	1	1	900
Dr. Sameer Kulkarni	Pediatrics	600	2	1	1	0	600
Dr. Imran Qureshi	General Medicine	400	2	1	1	0	400
Dr. Kavita Reddy	Neurology	1500	2	0	1	1	0

With `$1 = 3`, `HAVING` keeps only Dr. Priya Nair (4) and Dr. Arjun Singh (3).

CORRELATED SUBQUERY WITH NOT EXISTS (REPORT 2A: PATIENTS WITHOUT APPOINTMENTS)

For each patient, the inner query checks whether any appointment references them. It is **correlated** because it refers to `p.id` from the outer query.

```
SELECT p.id, p.name, p.age, p.phone
  FROM patients p
 WHERE NOT EXISTS (SELECT 1 FROM appointments a WHERE a.patient_id = p.id)
 ORDER BY p.name;
```

name	age	phone
Aditya Verma	29	9876500010
Neha Gupta	19	9876500005

NESTED SUBQUERY WITH AVG (REPORT 2B: PATIENTS OLDER THAN AVERAGE)

The inner query computes the average age once; the outer query keeps patients above it.

```
SELECT id, name, age,
       (SELECT ROUND(AVG(age), 1) FROM patients) AS avg_age
  FROM patients
 WHERE age > (SELECT AVG(age) FROM patients)
 ORDER BY age DESC;
```

Average age = 38.3. Result: Lakshmi Pillai (70), Vikram Joshi (61), Rahul Mehta (52), Karan Malhotra (45).

5.9 Query Coverage Checklist

Required operation	Where used
SELECT	All list screens, all reports
WHERE	Search on every tab; single-record fetch; upcoming appointments
ORDER BY	Every list; reports sorted by revenue, age, name
GROUP BY	Appointments by status; doctor workload
Aggregate functions	`COUNT`, `SUM`, `AVG`, `ROUND`
HAVING	Doctor workload `HAVING COUNT(a.id) >= $1`
Joins	Appointments list (`LEFT JOIN` x2), upcoming (`INNER JOIN` x2), workload (`LEFT JOIN`)
Nested queries / subqueries	Dashboard scalar subqueries, `NOT EXISTS` correlated subquery, `AVG` nested subquery

6. Sample Data

The guideline asks for at least 5-10 records per master table and 10-15 in transaction tables. The database is seeded with **10 patients, 6 doctors and 15 appointments**.

6.1 Patients (10 records, master table)

id	name	age	gender	phone	address
1	Anita Rao	34	Female	9876500001	12 MG Road, Bengaluru
2	Rahul Mehta	52	Male	9876500002	44 Park Street, Kolkata
3	Sara Khan	27	Female	9876500003	9 Marine Drive, Mumbai
4	Vikram Joshi	61	Male	9876500004	7 FC Road, Pune
5	Neha Gupta	19	Female	9876500005	21 Civil Lines, Jaipur
6	Karan Malhotra	45	Male	9876500006	3 Sector 17, Chandigarh
7	Fatima Sheikh	38	Female	9876500007	58 Banjara Hills, Hyderabad
8	Rohan Das	8	Male	9876500008	15 Salt Lake, Kolkata
9	Lakshmi Pillai	70	Female	9876500009	2 Anna Nagar, Chennai
10	Aditya Verma	29	Male	9876500010	66 Hazratganj, Lucknow

6.2 Doctors (6 records, master table)

id	name	specialty	phone	fee (₹)
1	Dr. Priya Nair	Cardiology	9811100001	1200
2	Dr. Arjun Singh	Orthopedics	9811100002	900
3	Dr. Meera Iyer	Dermatology	9811100003	700
4	Dr. Sameer Kulkarni	Pediatrics	9811100004	600
5	Dr. Kavita Reddy	Neurology	9811100005	1500
6	Dr. Imran Qureshi	General Medicine	9811100006	400

6.3 Appointments (15 records, transaction table)

id	patient	doctor	date	reason	status
1	Anita Rao	Dr. Priya Nair	2026-09-25	Chest pain follow-up	completed
2	Rahul Mehta	Dr. Arjun Singh	2026-09-26	Knee X-ray review	completed
3	Sara Khan	Dr. Meera Iyer	2026-09-27	Skin rash consult	completed
4	Vikram Joshi	Dr. Priya Nair	2026-10-02	ECG and BP check	completed
5	Rohan Das	Dr. Sameer Kulkarni	2026-10-05	Fever and cough	completed
6	Karan Malhotra	Dr. Imran Qureshi	2026-10-08	Annual health check	completed
7	Lakshmi Pillai	Dr. Kavita Reddy	2026-10-10	Migraine evaluation	cancelled
8	Rahul Mehta	Dr. Priya Nair	2026-10-14	Palpitations	completed
9	Fatima Sheikh	Dr. Meera Iyer	2026-10-18	Acne treatment	completed
10	Vikram Joshi	Dr. Arjun Singh	2026-10-21	Back pain	cancelled
11	Anita Rao	Dr. Priya Nair	2026-10-28	Medication review	scheduled
12	Lakshmi Pillai	Dr. Kavita Reddy	2026-10-30	Migraine follow-up	scheduled
13	Rohan Das	Dr. Sameer Kulkarni	2026-11-02	Vaccination	scheduled
14	Karan Malhotra	Dr. Arjun Singh	2026-11-04	Shoulder stiffness	scheduled
15	Fatima Sheikh	Dr. Imran Qureshi	2026-11-06	Blood test results	scheduled

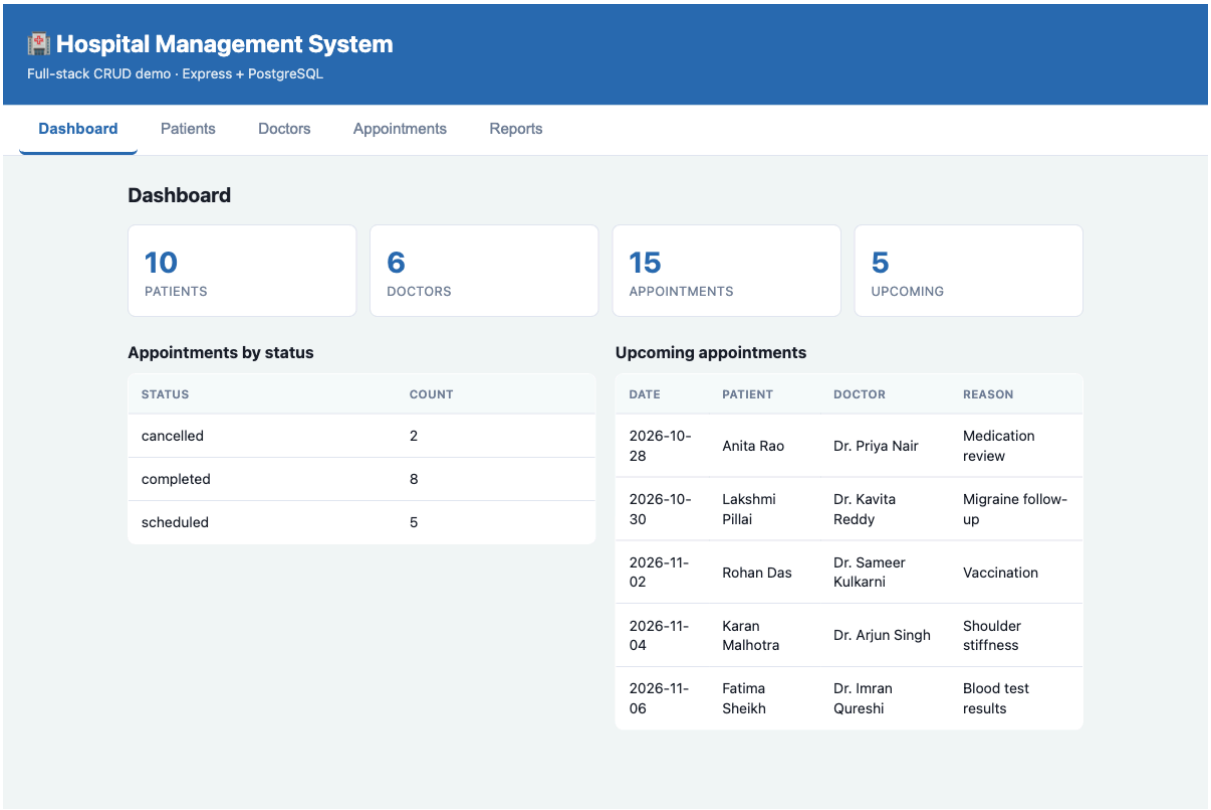
Patients 5 (Neha Gupta) and 10 (Aditya Verma) intentionally have no appointments so that the “patients without appointments” report has rows to show. Dates cover past (completed/cancelled) and future (scheduled) visits so the dashboard’s “upcoming” list is populated.

7. Frontend Prototype

The frontend is a single page (`public/index.html`) with five tabs. All data comes live from PostgreSQL through the REST API; nothing is hard-coded in the page.

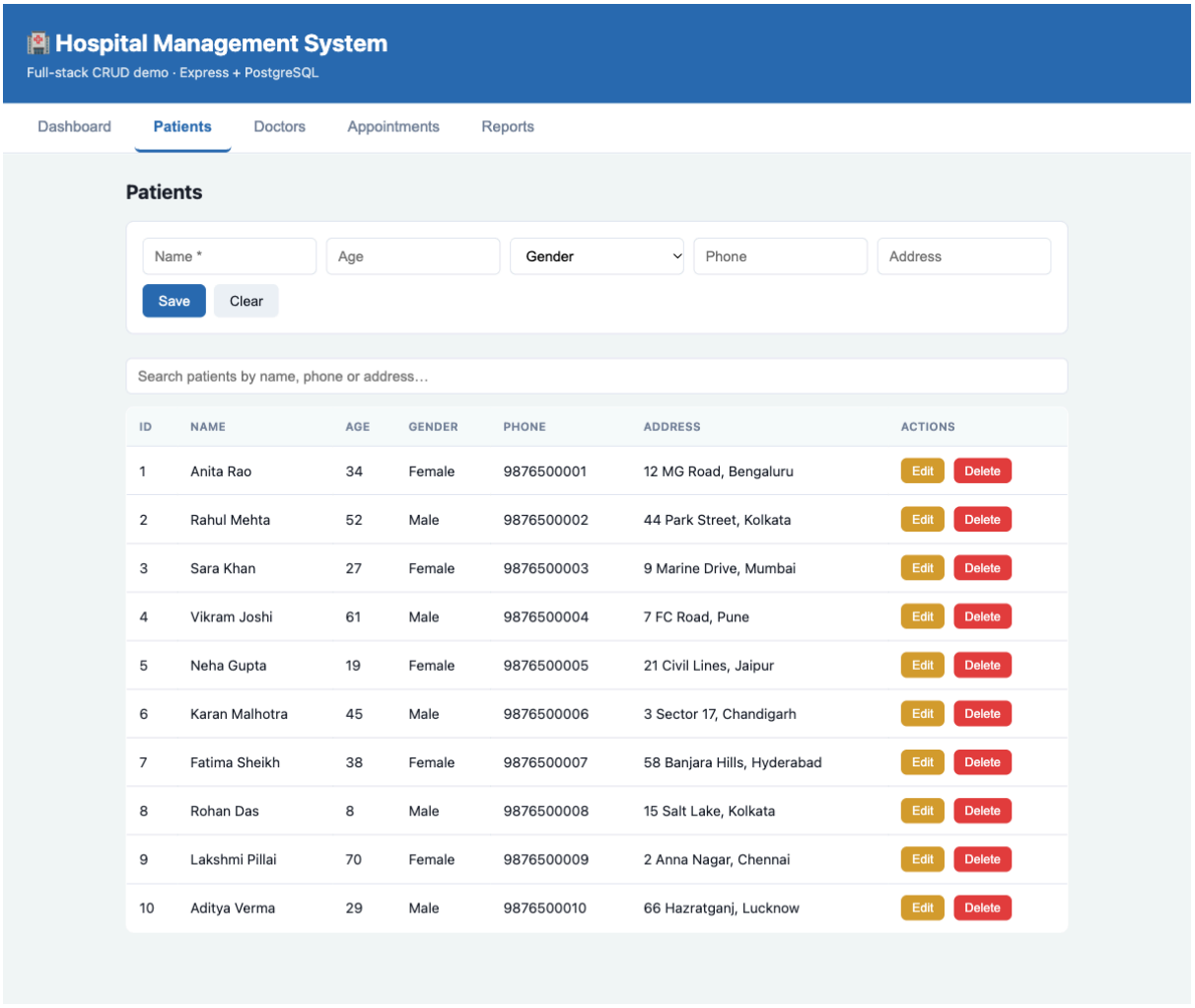
7.1 Home / Dashboard

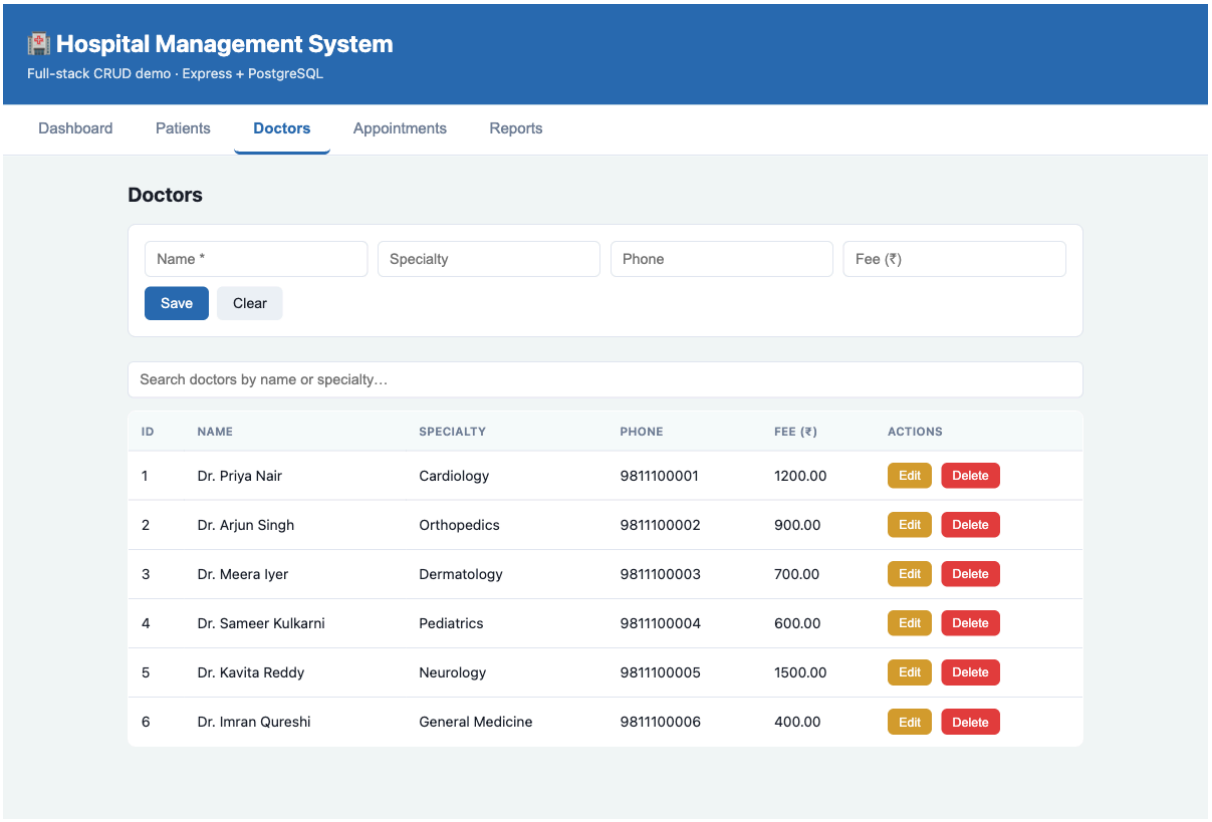
The first screen. Four stat cards (total patients, doctors, appointments, upcoming), a **GROUP BY** table of appointments per status, and the next five upcoming appointments (**JOIN**). It refreshes every time the tab is opened.

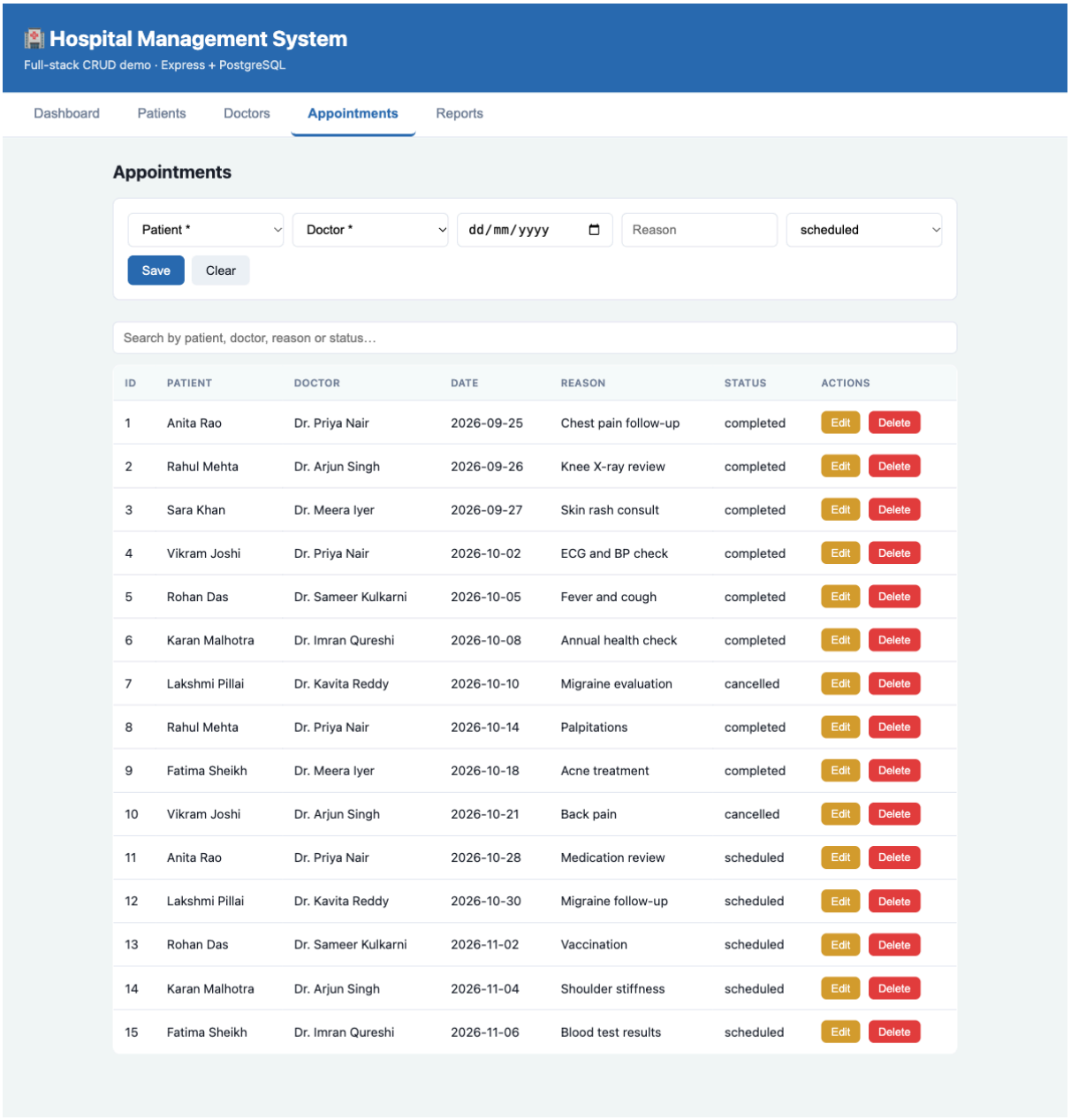


7.2 Add / Insert Record

Each entity tab has a form at the top. Leaving the hidden `id` empty and clicking **Save** sends a `POST` request, which runs an `INSERT`. Required fields are marked with `*` and also checked by the server and the database. The appointment form's Patient and Doctor dropdowns are filled from the database, so only valid foreign keys can be chosen.

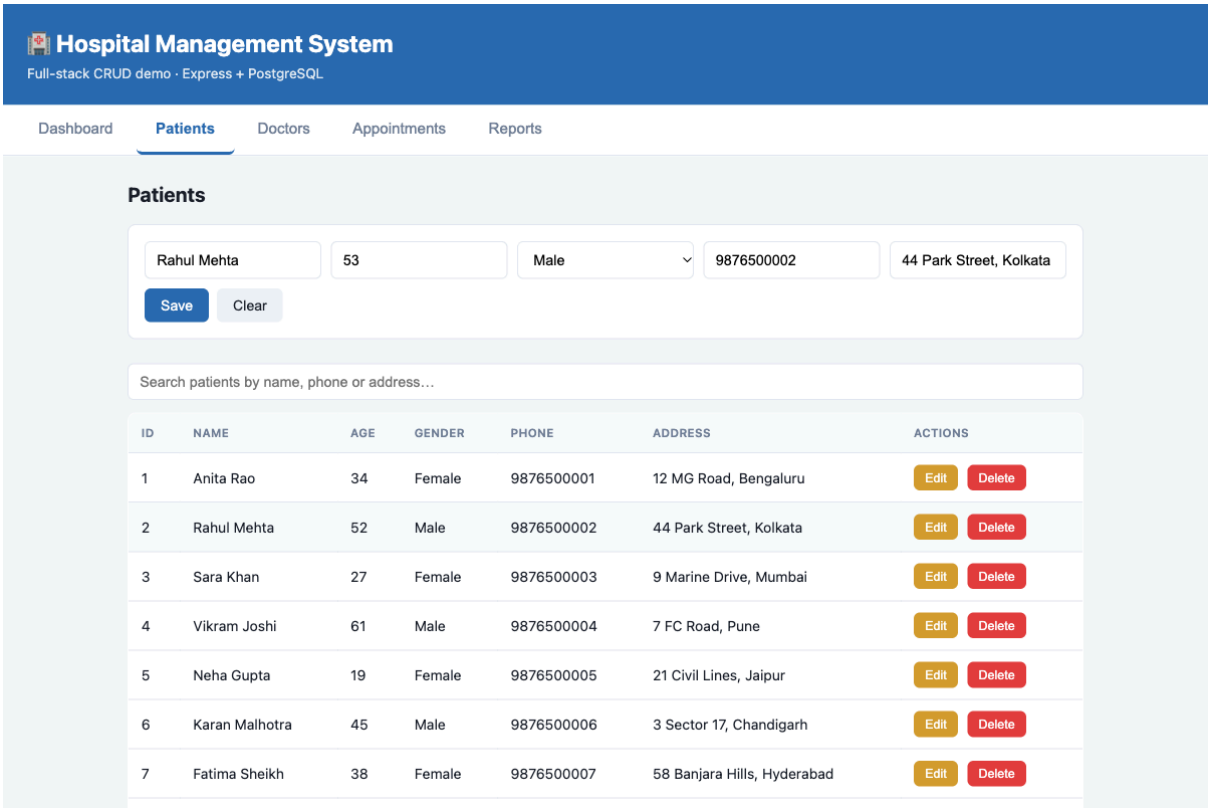

Hospital Management System
Full-stack CRUD demo · Express + PostgreSQL
Dashboard
Patients
Doctors
Appointments
Reports
Patients
Name *
Age
Gender
Phone
Address
Save
Clear
Search patients by name, phone or address…
ID NAME AGE GENDER PHONE ADDRESS ACTIONS
1 Anita Rao 34 Female 9876500001 12 MG Road, Bengaluru Edit Delete
2 Rahul Mehta 52 Male 9876500002 44 Park Street, Kolkata Edit Delete
3 Sara Khan 27 Female 9876500003 9 Marine Drive, Mumbai Edit Delete
4 Vikram Joshi 61 Male 9876500004 7 FC Road, Pune Edit Delete
5 Neha Gupta 19 Female 9876500005 21 Civil Lines, Jaipur Edit Delete
6 Karan Malhotra 45 Male 9876500006 3 Sector 17, Chandigarh Edit Delete
7 Fatima Sheikh 38 Female 9876500007 58 Banjara Hills, Hyderabad Edit Delete
8 Rohan Das 8 Male 9876500008 15 Salt Lake, Kolkata Edit Delete
9 Lakshmi Pillai 70 Female 9876500009 2 Anna Nagar, Chennai Edit Delete
10 Aditya Verma 29 Male 9876500010 66 Hazratganj, Lucknow Edit Delete


Hospital Management System
Full-stack CRUD demo · Express + PostgreSQL
Dashboard
Patients
Doctors
Appointments
Reports
Doctors
Name *
Specialty
Phone
Fee (₹)
Save
Clear
Search doctors by name or specialty…
ID
NAME
SPECIALTY
PHONE
FEE (₹)
ACTIONS
1
Dr. Priya Nair
Cardiology
9811100001
1200.00
Edit
Delete
2
Dr. Arjun Singh
Orthopedics
9811100002
900.00
Edit
Delete
3
Dr. Meera Iyer
Dermatology
9811100003
700.00
Edit
Delete
4
Dr. Sameer Kulkarni
Pediatrics
9811100004
600.00
Edit
Delete
5
Dr. Kavita Reddy
Neurology
9811100005
1500.00
Edit
Delete
6
Dr. Imran Qureshi
General Medicine
9811100006
400.00
Edit
Delete



7.3 Update Record

Clicking **Edit** on any row copies the row into the form (including its hidden `id`). Clicking **Save** then sends a `PUT` request, which runs an `UPDATE ... WHERE id = $n`. Below, Rahul Mehta's record has been loaded and his age is being changed from 52 to 53.

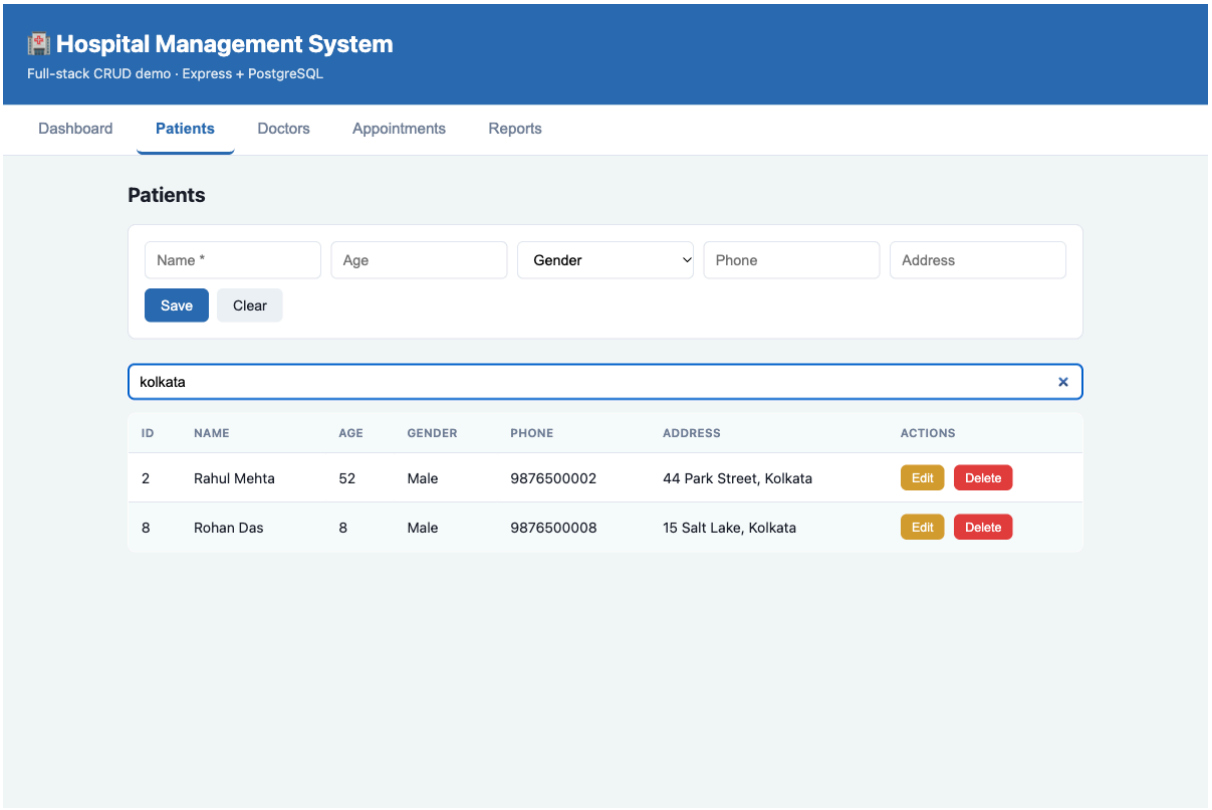


7.4 Delete Record

Clicking **Delete** asks for confirmation, then sends a `DELETE` request, which runs `DELETE FROM ... WHERE id = $1`. Deleting a patient or doctor also removes their appointments automatically because of `ON DELETE CASCADE`.

7.5 Search Record

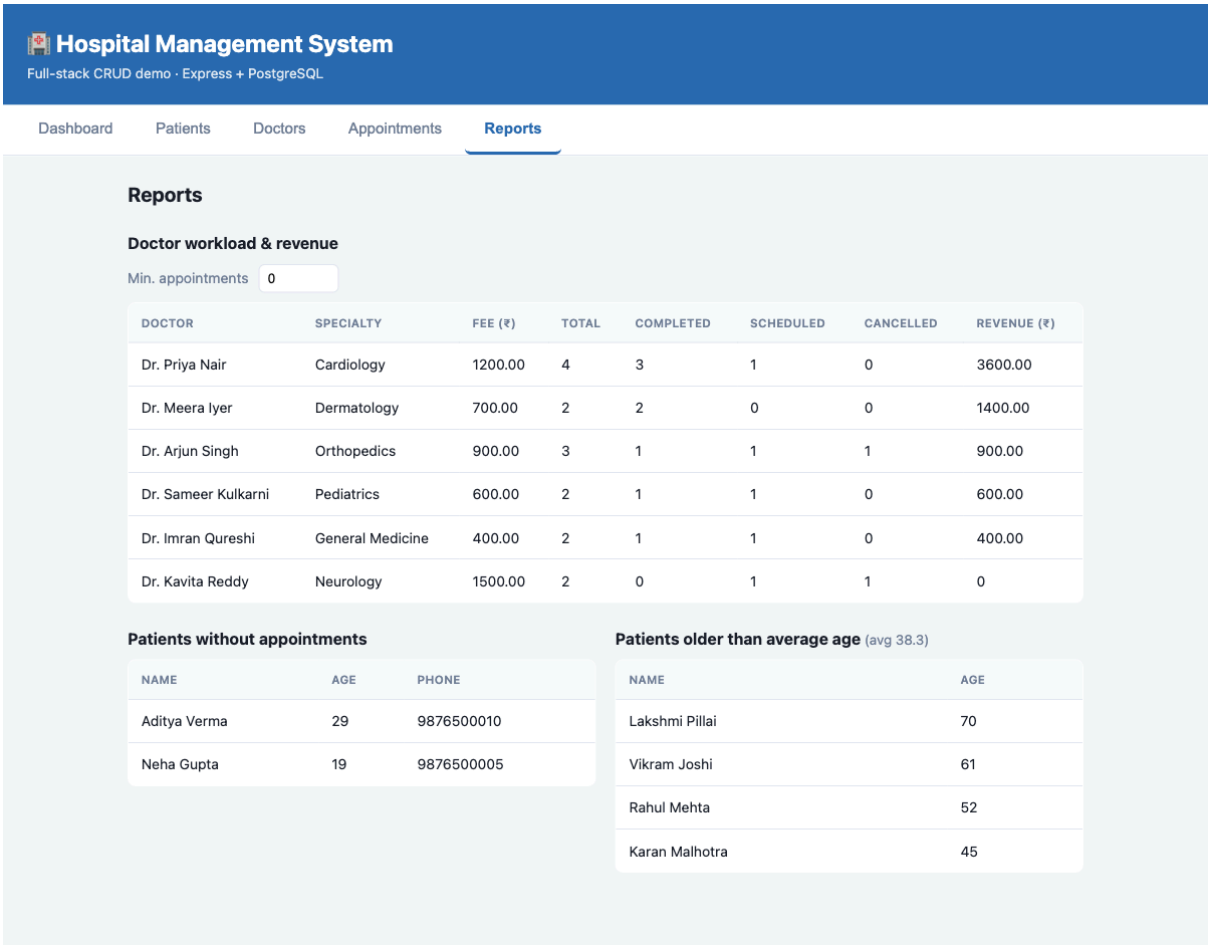
Every entity tab has a search box. Typing filters the table live (250 ms after the last keystroke) by sending `?q=<text>` to the API, which runs a `WHERE ... ILIKE` query. Patients are searched by name, phone or address; doctors by name or specialty; appointments by patient name, doctor name, reason or status. Below, searching “kolkata” returns the two patients with Kolkata addresses.



7.6 Reports

The Reports tab shows two reports:

1. **Doctor workload and revenue**: total, completed, scheduled and cancelled appointments per doctor, and revenue from completed visits (`JOIN`, `GROUP BY`, `COUNT`, `SUM`, `HAVING`).
2. **Patient reports**: patients who have never booked an appointment (`NOT EXISTS` subquery), and patients older than the average age (`AVG` nested subquery).



Changing **Min. appointments** to 3 re-runs the query with `HAVING COUNT(a.id) >= 3`, leaving only the two busiest doctors:

DOCTOR	SPECIALTY	FEE (₹)	TOTAL	COMPLETED	SCHEDULED	CANCELLED	REVENUE (₹)
Dr. Priya Nair	Cardiology	1200.00	4	3	1	0	3600.00
Dr. Arjun Singh	Orthopedics	900.00	3	1	1	1	900.00

7.7 Screens vs Guideline

Required screen/function	Implemented as
Home / Dashboard	Dashboard tab (default)
Add / Insert Record	Form + Save on Patients, Doctors, Appointments tabs
Update Record	Edit button, then Save
Delete Record	Delete button on every row
Search Record	Search box on every entity tab
Generate one or two Reports	Reports tab: doctor workload/revenue; patient reports

8. Backend: REST API

The Express server (`server.js`) exposes a JSON API under `/api` and serves the frontend from `/public`.

8.1 Endpoints

Method	Path	Action	SQL
GET	`/api/patients?q=`	List / search patients	`SELECT ... WHERE ... ILIKE ... ORDER BY`
GET	`/api/patients/:id`	Get one patient	`SELECT ... WHERE id = $1`
POST	`/api/patients`	Create patient	`INSERT ... RETURNING *`
PUT	`/api/patients/:id`	Update patient	`UPDATE ... WHERE id = $n`
DELETE	`/api/patients/:id`	Delete patient	`DELETE ... WHERE id = $1`
GET/POST/PUT/DELETE	`/api/doctors[/:id]`	Same for doctors	same pattern
GET/POST/PUT/DELETE	`/api/appointments[/:id]`	Same for appointments (list includes joined names)	same pattern + `JOIN`
GET	`/api/reports/summary`	Dashboard data	scalar subqueries, `GROUP BY`, `JOIN`
GET	`/api/reports/doctor-workload?min=N`	Report 1	`LEFT JOIN`, `GROUP BY`, `HAVING`
GET	`/api/reports/patients`	Report 2	`NOT EXISTS`, `AVG` subquery
GET	`/api/health`	Health check	none

8.2 Response Codes

Code	When
200 OK	Successful read, update or delete
201 Created	Successful insert
400 Bad Request	Missing required field (name; or patient, doctor and date for appointments)
404 Not Found	No row with the given id
500 Server Error	Database error, e.g. a constraint violation. The PostgreSQL error message is returned as `{ "error": "..." }`.

8.3 Example

```
$ curl localhost:3000/api/doctors?q=neuro
[{"id":5,"name":"Dr. Kavita Reddy","specialty":"Neurology",
  "phone":"9811100005","consultation_fee":"1500.00"}]
```

9. Code Walkthrough

9.1 `db.js`: database connection

Creates one shared **connection pool** from `DATABASE_URL`. A pool keeps a few connections open and reuses them, so each request does not pay the cost of opening a new connection. It also registers a type parser so PostgreSQL `DATE` values (type OID 1082) are returned as plain `'YYYY-MM-DD'` strings. Without this, the driver converts dates to JavaScript `Date` objects at local midnight, and converting those to UTC can shift the date back by one day in time zones ahead of UTC (such as India, UTC+5:30).

9.2 `server.js`: application entry point

- `express.json()` parses JSON request bodies.
- `express.static('public')` serves the HTML, CSS and JS files.
- Mounts the four routers at `/api/patients`, `/api/doctors`, `/api/appointments`, `/api/reports`.
- Listens on `PORT` (default 3000).

9.3 `routes/patients.js`, `doctors.js`, `appointments.js`: CRUD

Each file defines five handlers: list (with optional search), get one, create, update and delete. Every handler:

1. Reads input from `req.params`, `req.query` or `req.body`.
2. Validates required fields and returns 400 if missing.
3. Runs **one parameterised SQL statement** through `db.query(sql, params)`.
4. Returns 404 if no row was affected, otherwise the row(s) as JSON.
5. Catches database errors and returns them as JSON with status 500.

`appointments.js` defines a shared `LIST_SQL` with the two `LEFT JOIN`s so list and get-one return `patient_name` and `doctor_name`.

9.4 `routes/reports.js`: dashboard and reports

Read-only endpoints. `/summary` runs its three queries in parallel with `Promise.all`. `/doctor-workload` reads `?min=` and passes it as the `HAVING` parameter. `/patients` runs the `NOT EXISTS` and `AVG` subquery reports in parallel.

9.5 `public/app.js`: frontend logic

- **`api(path, options)`**: wrapper around `fetch('/api' + path)` that parses JSON and throws on error.
- **`toast(msg)`**: shows a short success or error message at the bottom of the screen.
- **`esc(value)`**: HTML-escapes every value before it is inserted into the page, preventing cross-site scripting (XSS) if a name contains `<script>`.
- **Tab switching**: shows the selected panel; reloads the dashboard, reports or appointment dropdowns when their tab opens so they reflect changes made elsewhere.
- **`makeEntity({...})`**: one generic controller reused for patients, doctors and appointments. It loads and renders the table, wires Edit (fills the form) and Delete (confirm then `DELETE`), handles Save (`POST` if no id, `PUT` if id is set), and debounces the search box.
- **`fillAppointmentSelects()`**: fills the Patient and Doctor dropdowns from the database.
- **`loadDashboard()` / `loadReports()`**: fetch the report endpoints and render them with the `fillTable()` helper.

9.6 `public/index.html` and `style.css`

`index.html` contains the header, tab bar and five `<section class="panel">` blocks (Dashboard, Patients, Doctors, Appointments, Reports). Each entity panel has a form, a search box and a table. `style.css` defines the colour variables, tab styling, card grid for dashboard stats, responsive two-column grid for reports, and table styling.

9.7 Security Measures

Risk	Mitigation
SQL injection	All user input goes through `$1, $2, ...` parameters; the `pg` driver sends values separately from the SQL text.
Cross-site scripting (XSS)	Every value is passed through `esc()` before being written into HTML.
Invalid data	Validated in three layers: HTML `required` / `min` attributes, server checks, and database constraints (the final authority).
Credentials in source	Database URL lives in `.env`, which is listed in `.gitignore`.

10. Installation and Running

10.1 Prerequisites

- Node.js 20 or newer
- PostgreSQL 18 running locally (`brew services start postgresql@18` on macOS)
- pgAdmin (optional)

10.2 Steps

```
# 1. Install dependencies
npm install

# 2. Create the database, tables and sample data
#    (re-runnable; drops and recreates the tables)
npm run initdb            # = psql -d postgres -f schema.sql

# 3. Configure the connection
cp .env.example .env
#    DATABASE_URL=postgresql://localhost:5432/hospital
#    PORT=3000

# 4. Start the server
npm start                 # open http://localhost:3000
```

On Homebrew PostgreSQL the database user is the macOS username with no password. Otherwise use `postgresql://USER:PASSWORD@localhost:5432/hospital`.

10.3 Viewing Data in pgAdmin

Connect with host `localhost`, port `5432`, database `hospital`, user = your OS username. Open *Schemas* → *public* → *Tables* and use *View/Edit Data* to watch rows change as you use the app.

11. Testing

#	Test	Expected	Result
1	Load schema into a fresh database	10 / 6 / 15 rows	Pass
2	Open Dashboard	Totals 10/6/15/5, status breakdown, 5 upcoming	Pass
3	Add a doctor without fee	Fee defaults to 500.00	Pass
4	Update that doctor’s fee to 800	Fee becomes 800.00	Pass
5	Add patient with duplicate phone	Rejected by `UNIQUE`	Pass
6	Add patient with negative age	Rejected by `CHECK`	Pass
7	Add appointment without date	Rejected (400)	Pass
8	Search patients “kol”	Rahul Mehta, Rohan Das	Pass
9	Search appointments “migraine”	Appointments 7 and 12	Pass
10	Search doctors “neuro”	Dr. Kavita Reddy	Pass
11	Workload report with min = 3	Dr. Priya Nair, Dr. Arjun Singh	Pass
12	Patients without appointments	Aditya Verma, Neha Gupta	Pass
13	Patients above average age (38.3)	4 patients, oldest first	Pass
14	Browser console	No JavaScript errors	Pass

12. Guideline Compliance Summary

Guideline section	Requirement	Status	Section of this report
1. Problem statement	Title, background, existing problem, proposed solution	Met	1
2. ER diagram	Entities, attributes, PKs, relationships, cardinality, participation	Met	2
3. Relational model	Tables, attributes, PK, FK, candidate keys, data types, constraints	Met	3
4. DDL	CREATE DATABASE, CREATE/ALTER TABLE, constraints	Met	5.1 - 5.4
4. DML	INSERT, UPDATE, DELETE	Met	5.5 - 5.7
4. Queries	SELECT, WHERE, ORDER BY, GROUP BY, aggregates, HAVING, joins, subqueries	Met	5.8 - 5.9
5. Sample data	5-10 per master table, 10-15 per transaction table	Met (10, 6, 15)	6
6. Frontend	Dashboard, Add, Update, Delete, Search, Reports	Met	7

13. Limitations and Future Scope

Current limitations

- No login or user roles; anyone who can open the page can edit data.
- Appointments have a date but no time slot.
- Revenue in the workload report uses each doctor's **current** fee. If a fee changes, past revenue is recalculated at the new rate. A production system would store the fee charged on each appointment.
- Deleting a patient or doctor cascades to their appointment history; a production system would more likely mark records inactive instead.

Future enhancements

- Authentication with receptionist, doctor and admin roles.
- Time slots and a doctor availability calendar with overlap checks.
- Billing and invoice tables linked to completed appointments.
- Prescriptions and medical history per appointment.
- Export reports to CSV/PDF; charts on the dashboard.
- Database indexes on `appointments(doctor_id)`, `appointments(patient_id)` and `appointments(appt_date)` as data grows.

14. Conclusion

The Hospital Management System applies the full DBMS workflow to a real-world problem: a clear problem statement, an ER model with cardinality and participation, a 3NF relational schema with primary, foreign and candidate keys, and a PostgreSQL implementation that uses DDL, DML and every required query type. Data integrity is enforced by the database itself through constraints and cascading foreign keys. A working web frontend demonstrates insert, update, delete, search, a live dashboard and two management reports against realistic sample data, meeting every point of the micro project guidelines.

Appendix: Complete Source Code

schema.sql

```
-- Hospital Management System schema + seed data.
-- Run: psql -d postgres -f schema.sql      (or: npm run initdb)
-- Re-runnable: creates the database if missing, then drops + recreates tables.

-- DDL: CREATE DATABASE ------------------------------------------------
-- CREATE DATABASE has no IF NOT EXISTS in PostgreSQL, so build it
-- conditionally and execute it with psql's \gexec.
SELECT 'CREATE DATABASE hospital'
 WHERE NOT EXISTS (SELECT FROM pg_database WHERE datname = 'hospital')\gexec

\c hospital

-- Dropping in FK-safe order so the script is re-runnable.
DROP TABLE IF EXISTS appointments;
DROP TABLE IF EXISTS patients;
DROP TABLE IF EXISTS doctors;

-- DDL: CREATE TABLE ---------------------------------------------------

CREATE TABLE patients (
  id      SERIAL       PRIMARY KEY,
  name    VARCHAR(100) NOT NULL,
  age     INT          CHECK (age BETWEEN 0 AND 150),
  gender  VARCHAR(10)  CHECK (gender IN ('Male', 'Female', 'Other')),
  phone   VARCHAR(15)  UNIQUE,                -- candidate key
  address VARCHAR(255)
);

CREATE TABLE doctors (
  id        SERIAL       PRIMARY KEY,
  name      VARCHAR(100) NOT NULL,
  specialty VARCHAR(50),
  phone     VARCHAR(15)  UNIQUE               -- candidate key
);

CREATE TABLE appointments (
  id         SERIAL      PRIMARY KEY,
  patient_id INT         NOT NULL REFERENCES patients(id) ON DELETE CASCADE,
  doctor_id  INT         NOT NULL REFERENCES doctors(id)  ON DELETE CASCADE,
  appt_date  DATE        NOT NULL,
  reason     VARCHAR(255),
  status     VARCHAR(20) NOT NULL DEFAULT 'scheduled'
                         CHECK (status IN ('scheduled', 'completed',
'cancelled'))
);

-- DDL: ALTER TABLE ----------------------------------------------------
```

```
ALTER TABLE doctors
  ADD COLUMN consultation_fee NUMERIC(8, 2) NOT NULL DEFAULT 500
             CHECK (consultation_fee >= 0);

-- A doctor cannot be booked twice by the same patient on the same day.
ALTER TABLE appointments
  ADD CONSTRAINT uq_patient_doctor_date UNIQUE (patient_id, doctor_id,
appt_date);

-- DML: seed data -------------------------------------------------------

INSERT INTO patients (name, age, gender, phone, address) VALUES
  ('Anita Rao',      34, 'Female', '9876500001', '12 MG Road, Bengaluru'),
  ('Rahul Mehta',    52, 'Male',   '9876500002', '44 Park Street, Kolkata'),
  ('Sara Khan',      27, 'Female', '9876500003', '9 Marine Drive, Mumbai'),
  ('Vikram Joshi',   61, 'Male',   '9876500004', '7 FC Road, Pune'),
  ('Neha Gupta',     19, 'Female', '9876500005', '21 Civil Lines, Jaipur'),
  ('Karan Malhotra', 45, 'Male',   '9876500006', '3 Sector 17, Chandigarh'),
  ('Fatima Sheikh',  38, 'Female', '9876500007', '58 Banjara Hills,
Hyderabad'),
  ('Rohan Das',      8,  'Male',   '9876500008', '15 Salt Lake, Kolkata'),
  ('Lakshmi Pillai', 70, 'Female', '9876500009', '2 Anna Nagar, Chennai'),
  ('Aditya Verma',   29, 'Male',   '9876500010', '66 Hazratganj, Lucknow');

INSERT INTO doctors (name, specialty, phone, consultation_fee) VALUES
  ('Dr. Priya Nair',    'Cardiology',  '9811100001', 1200),
  ('Dr. Arjun Singh',   'Orthopedics', '9811100002', 900),
  ('Dr. Meera Iyer',    'Dermatology', '9811100003', 700),
  ('Dr. Sameer Kulkarni','Pediatrics', '9811100004', 600),
  ('Dr. Kavita Reddy',  'Neurology',   '9811100005', 1500),
  ('Dr. Imran Qureshi', 'General Medicine', '9811100006', 400);

INSERT INTO appointments (patient_id, doctor_id, appt_date, reason, status)
VALUES
  (1, 1, '2026-09-25', 'Chest pain follow-up',     'completed'),
  (2, 2, '2026-09-26', 'Knee X-ray review',        'completed'),
  (3, 3, '2026-09-27', 'Skin rash consult',        'completed'),
  (4, 1, '2026-10-02', 'ECG and BP check',         'completed'),
  (8, 4, '2026-10-05', 'Fever and cough',          'completed'),
  (6, 6, '2026-10-08', 'Annual health check',      'completed'),
  (9, 5, '2026-10-10', 'Migraine evaluation',      'cancelled'),
  (2, 1, '2026-10-14', 'Palpitations',             'completed'),
  (7, 3, '2026-10-18', 'Acne treatment',           'completed'),
  (4, 2, '2026-10-21', 'Back pain',                'cancelled'),
  (1, 1, '2026-10-28', 'Medication review',        'scheduled'),
  (9, 5, '2026-10-30', 'Migraine follow-up',       'scheduled'),
  (8, 4, '2026-11-02', 'Vaccination',              'scheduled'),
  (6, 2, '2026-11-04', 'Shoulder stiffness',       'scheduled'),
  (7, 6, '2026-11-06', 'Blood test results',       'scheduled');

-- Patients 5 (Neha) and 10 (Aditya) intentionally have no appointments,
-- so the "patients without appointments" report has rows to show.
```

db.js

```
// PostgreSQL connection pool. All routes query through here.
require('dotenv').config();
const { Pool, types } = require('pg');

// Return DATE columns as plain 'YYYY-MM-DD' strings. The default parses them
// into a local-midnight JS Date, which shifts the day when serialized to UTC.
types.setTypeParser(1082, (v) => v);

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
});

pool.on('error', (err) => {
  console.error('Unexpected PG pool error:', err.message);
});

module.exports = {
  query: (text, params) => pool.query(text, params),
  pool,
};
```

server.js

```
// Express server: JSON REST API + static frontend.
require('dotenv').config();
const path = require('path');
const express = require('express');

const app = express();

app.use(express.json());
app.use(express.static(path.join(__dirname, 'public')));

// API routes
app.use('/api/patients', require('./routes/patients'));
app.use('/api/doctors', require('./routes/doctors'));
app.use('/api/appointments', require('./routes/appointments'));
app.use('/api/reports', require('./routes/reports'));

// Health check
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Hospital CRUD running at http://localhost:${PORT}`);
});
```

package.json

```
{
  "name": "dbms-crud",
  "version": "1.0.0",
  "description": "Hospital Management System - full-stack CRUD (Express +
PostgreSQL)",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "node --watch server.js",
    "initdb": "psql -d postgres -f schema.sql"
  },
  "dependencies": {
    "dotenv": "^16.4.5",
    "express": "^4.21.2",
    "pg": "^8.13.1"
  }
}
```

.env.example

```
# Copy to .env and edit if your Postgres setup differs.
# Homebrew Postgres default: no password, user = your macOS username.
DATABASE_URL=postgresql://localhost:5432/hospital
PORT=3000
```

routes/patients.js

```
// CRUD routes for patients.
const express = require('express');
const db = require('../db');

const router = express.Router();

// READ all (optional ?q= search on name / phone / address)
router.get('/', async (req, res) => {
  try {
    const { rows } = await db.query(
      `SELECT * FROM patients
       WHERE $1::text IS NULL
          OR name ILIKE '%' || $1 || '%'
          OR phone ILIKE '%' || $1 || '%'
          OR address ILIKE '%' || $1 || '%'
       ORDER BY id`,
      [req.query.q || null]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// READ one
router.get('/:id', async (req, res) => {
  try {
    const { rows } = await db.query('SELECT * FROM patients WHERE id = $1',
[req.params.id]);
    if (rows.length === 0) return res.status(404).json({ error: 'Patient not
found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// CREATE
router.post('/', async (req, res) => {
  const { name, age, gender, phone, address } = req.body;
  if (!name) return res.status(400).json({ error: 'name is required' });
  try {
    const { rows } = await db.query(
      `INSERT INTO patients (name, age, gender, phone, address)
       VALUES ($1, $2, $3, $4, $5) RETURNING *`,
      [name, age || null, gender || null, phone || null, address || null]
    );
    res.status(201).json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
// UPDATE
router.put('/:id', async (req, res) => {
  const { name, age, gender, phone, address } = req.body;
  try {
    const { rows } = await db.query(
      `UPDATE patients
         SET name = $1, age = $2, gender = $3, phone = $4, address = $5
       WHERE id = $6 RETURNING *`,
      [name, age || null, gender || null, phone || null, address || null,
req.params.id]
    );
    if (rows.length === 0) return res.status(404).json({ error: 'Patient not
found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const { rowCount } = await db.query('DELETE FROM patients WHERE id = $1',
[req.params.id]);
    if (rowCount === 0) return res.status(404).json({ error: 'Patient not
found' });
    res.json({ message: 'Patient deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

routes/doctors.js

```
// CRUD routes for doctors.
const express = require('express');
const db = require('../db');

const router = express.Router();

// READ all (optional ?q= search on name / specialty)
router.get('/', async (req, res) => {
  try {
    const { rows } = await db.query(
      `SELECT * FROM doctors
       WHERE $1::text IS NULL
          OR name ILIKE '%' || $1 || '%'
          OR specialty ILIKE '%' || $1 || '%'
       ORDER BY id`,
      [req.query.q || null]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// READ one
router.get('/:id', async (req, res) => {
  try {
    const { rows } = await db.query('SELECT * FROM doctors WHERE id = $1',
[req.params.id]);
    if (rows.length === 0) return res.status(404).json({ error: 'Doctor not
found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// CREATE
router.post('/', async (req, res) => {
  const { name, specialty, phone, consultation_fee } = req.body;
  if (!name) return res.status(400).json({ error: 'name is required' });
  try {
    const { rows } = await db.query(
      `INSERT INTO doctors (name, specialty, phone, consultation_fee)
       VALUES ($1, $2, $3, COALESCE($4, 500)) RETURNING *`,
      [name, specialty || null, phone || null, consultation_fee ?? null]
    );
    res.status(201).json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
```

```
// UPDATE
router.put('/:id', async (req, res) => {
  const { name, specialty, phone, consultation_fee } = req.body;
  try {
    const { rows } = await db.query(
      `UPDATE doctors
         SET name = $1, specialty = $2, phone = $3,
             consultation_fee = COALESCE($4, consultation_fee)
       WHERE id = $5 RETURNING *`,
      [name, specialty || null, phone || null, consultation_fee ?? null,
req.params.id]
    );
    if (rows.length === 0) return res.status(404).json({ error: 'Doctor not
found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const { rowCount } = await db.query('DELETE FROM doctors WHERE id = $1',
[req.params.id]);
    if (rowCount === 0) return res.status(404).json({ error: 'Doctor not found'
});
    res.json({ message: 'Doctor deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

routes/appointments.js

```
// CRUD routes for appointments. List uses JOINs to show patient + doctor
names.
const express = require('express');
const db = require('../db');

const router = express.Router();

const LIST_SQL = `
  SELECT a.id, a.patient_id, a.doctor_id, a.appt_date, a.reason, a.status,
         p.name AS patient_name,
         d.name AS doctor_name
    FROM appointments a
    LEFT JOIN patients p ON p.id = a.patient_id
    LEFT JOIN doctors  d ON d.id = a.doctor_id
`;

// READ all (with joined names; optional ?q= search on names / reason / status)
router.get('/', async (req, res) => {
  try {
    const { rows } = await db.query(
      `${LIST_SQL}
        WHERE $1::text IS NULL
           OR p.name ILIKE '%' || $1 || '%'
           OR d.name ILIKE '%' || $1 || '%'
           OR a.reason ILIKE '%' || $1 || '%'
           OR a.status ILIKE '%' || $1 || '%'
        ORDER BY a.id`,
      [req.query.q || null]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// READ one
router.get('/:id', async (req, res) => {
  try {
    const { rows } = await db.query(`${LIST_SQL} WHERE a.id = $1`,
[req.params.id]);
    if (rows.length === 0) return res.status(404).json({ error: 'Appointment
not found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// CREATE
router.post('/', async (req, res) => {
  const { patient_id, doctor_id, appt_date, reason, status } = req.body;
  if (!patient_id || !doctor_id || !appt_date) {
```

```
    return res.status(400).json({ error: 'patient_id, doctor_id and appt_date
are required' });
  }
  try {
    const { rows } = await db.query(
      `INSERT INTO appointments (patient_id, doctor_id, appt_date, reason,
status)
       VALUES ($1, $2, $3, $4, $5) RETURNING *`,
      [patient_id, doctor_id, appt_date || null, reason || null, status ||
'scheduled']
    );
    res.status(201).json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// UPDATE
router.put('/:id', async (req, res) => {
  const { patient_id, doctor_id, appt_date, reason, status } = req.body;
  try {
    const { rows } = await db.query(
      `UPDATE appointments
         SET patient_id = $1, doctor_id = $2, appt_date = $3, reason = $4,
status = $5
       WHERE id = $6 RETURNING *`,
      [patient_id, doctor_id, appt_date || null, reason || null, status ||
'scheduled', req.params.id]
    );
    if (rows.length === 0) return res.status(404).json({ error: 'Appointment
not found' });
    res.json(rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE
router.delete('/:id', async (req, res) => {
  try {
    const { rowCount } = await db.query('DELETE FROM appointments WHERE id =
$1', [req.params.id]);
    if (rowCount === 0) return res.status(404).json({ error: 'Appointment not
found' });
    res.json({ message: 'Appointment deleted' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

routes/reports.js

```
// Read-only dashboard + report queries (aggregates, GROUP BY, HAVING,
subqueries).
const express = require('express');
const db = require('../db');

const router = express.Router();

// Dashboard: totals (scalar subqueries), status breakdown (GROUP BY), upcoming
list.
router.get('/summary', async (req, res) => {
  try {
    const [totals, byStatus, upcoming] = await Promise.all([
      db.query(`
        SELECT (SELECT COUNT(*) FROM patients)::int     AS patients,
               (SELECT COUNT(*) FROM doctors)::int      AS doctors,
               (SELECT COUNT(*) FROM appointments)::int AS appointments,
               (SELECT COUNT(*) FROM appointments
                 WHERE status = 'scheduled' AND appt_date >= CURRENT_DATE)::int
AS upcoming
      `),
      db.query(`
        SELECT status, COUNT(*)::int AS count
          FROM appointments
         GROUP BY status
         ORDER BY status
      `),
      db.query(`
        SELECT a.id, a.appt_date, a.reason, p.name AS patient_name, d.name AS
doctor_name
          FROM appointments a
          JOIN patients p ON p.id = a.patient_id
          JOIN doctors  d ON d.id = a.doctor_id
         WHERE a.status = 'scheduled' AND a.appt_date >= CURRENT_DATE
         ORDER BY a.appt_date
         LIMIT 5
      `),
    ]);
    res.json({ totals: totals.rows[0], byStatus: byStatus.rows, upcoming:
upcoming.rows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Report 1: doctor workload + revenue. JOIN, GROUP BY, COUNT/SUM, HAVING.
// ?min=N keeps only doctors with at least N appointments.
router.get('/doctor-workload', async (req, res) => {
  const min = Number.parseInt(req.query.min, 10) || 0;
  try {
    const { rows } = await db.query(
      `SELECT d.id, d.name, d.specialty, d.consultation_fee,
              COUNT(a.id)::int                                          AS
```

```
total,
              COUNT(a.id) FILTER (WHERE a.status = 'completed')::int   AS
completed,
              COUNT(a.id) FILTER (WHERE a.status = 'scheduled')::int   AS
scheduled,
              COUNT(a.id) FILTER (WHERE a.status = 'cancelled')::int   AS
cancelled,
              COALESCE(SUM(d.consultation_fee) FILTER (WHERE a.status =
'completed'), 0) AS revenue
         FROM doctors d
         LEFT JOIN appointments a ON a.doctor_id = d.id
        GROUP BY d.id
       HAVING COUNT(a.id) >= $1
        ORDER BY revenue DESC, total DESC`,
      [min]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Report 2: patients with no appointments (NOT EXISTS subquery) and
// patients older than the average age (nested aggregate subquery).
router.get('/patients', async (req, res) => {
  try {
    const [inactive, aboveAvg] = await Promise.all([
      db.query(`
        SELECT p.id, p.name, p.age, p.phone
          FROM patients p
         WHERE NOT EXISTS (SELECT 1 FROM appointments a WHERE a.patient_id =
p.id)
         ORDER BY p.name
      `),
      db.query(`
        SELECT id, name, age,
               (SELECT ROUND(AVG(age), 1) FROM patients) AS avg_age
          FROM patients
         WHERE age > (SELECT AVG(age) FROM patients)
         ORDER BY age DESC
      `),
    ]);
    res.json({ inactive: inactive.rows, aboveAvg: aboveAvg.rows });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

module.exports = router;
```

public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Hospital Management System</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <header>
    <h1>🏥 Hospital Management System</h1>
    <p>Full-stack CRUD demo · Express + PostgreSQL</p>
  </header>

  <nav class="tabs">
    <button class="tab active" data-tab="dashboard">Dashboard</button>
    <button class="tab" data-tab="patients">Patients</button>
    <button class="tab" data-tab="doctors">Doctors</button>
    <button class="tab" data-tab="appointments">Appointments</button>
    <button class="tab" data-tab="reports">Reports</button>
  </nav>

  <!-- DASHBOARD -->
  <section id="dashboard" class="panel active">
    <h2>Dashboard</h2>
    <div class="stats" id="stats"></div>
    <div class="grid-2">
      <div>
        <h3>Appointments by status</h3>
        <table id="status-table">
          <thead><tr><th>Status</th><th>Count</th></tr></thead>
          <tbody></tbody>
        </table>
      </div>
      <div>
        <h3>Upcoming appointments</h3>
        <table id="upcoming-table">
          <thead><tr><th>Date</th><th>Patient</th><th>Doctor</th>
<th>Reason</th></tr></thead>
          <tbody></tbody>
        </table>
      </div>
    </div>
  </section>

  <!-- PATIENTS -->
  <section id="patients" class="panel">
    <h2>Patients</h2>
    <form id="patient-form" class="crud-form">
      <input type="hidden" name="id" />
      <input name="name" placeholder="Name *" required />
      <input name="age" type="number" placeholder="Age" min="0" />
```

```
      <select name="gender">
        <option value="">Gender</option>
        <option>Male</option>
        <option>Female</option>
        <option>Other</option>
      </select>
      <input name="phone" placeholder="Phone" />
      <input name="address" placeholder="Address" />
      <div class="form-actions">
        <button type="submit" class="btn-primary">Save</button>
        <button type="button" class="btn-reset" data-reset>Clear</button>
      </div>
    </form>
    <input type="search" class="search" data-search="patients"
placeholder="Search patients by name, phone or address…" />
    <table id="patients-table">
      <thead>
        <tr><th>ID</th><th>Name</th><th>Age</th><th>Gender</th><th>Phone</th>
<th>Address</th><th>Actions</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </section>

  <!-- DOCTORS -->
  <section id="doctors" class="panel">
    <h2>Doctors</h2>
    <form id="doctor-form" class="crud-form">
      <input type="hidden" name="id" />
      <input name="name" placeholder="Name *" required />
      <input name="specialty" placeholder="Specialty" />
      <input name="phone" placeholder="Phone" />
      <input name="consultation_fee" type="number" min="0" step="0.01"
placeholder="Fee (₹)" />
      <div class="form-actions">
        <button type="submit" class="btn-primary">Save</button>
        <button type="button" class="btn-reset" data-reset>Clear</button>
      </div>
    </form>
    <input type="search" class="search" data-search="doctors"
placeholder="Search doctors by name or specialty…" />
    <table id="doctors-table">
      <thead>
        <tr><th>ID</th><th>Name</th><th>Specialty</th><th>Phone</th><th>Fee (₹)
</th><th>Actions</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </section>

  <!-- APPOINTMENTS -->
  <section id="appointments" class="panel">
    <h2>Appointments</h2>
    <form id="appointment-form" class="crud-form">
      <input type="hidden" name="id" />
```

```
      <select name="patient_id" required>
        <option value="">Patient *</option>
      </select>
      <select name="doctor_id" required>
        <option value="">Doctor *</option>
      </select>
      <input name="appt_date" type="date" required />
      <input name="reason" placeholder="Reason" />
      <select name="status">
        <option value="scheduled">scheduled</option>
        <option value="completed">completed</option>
        <option value="cancelled">cancelled</option>
      </select>
      <div class="form-actions">
        <button type="submit" class="btn-primary">Save</button>
        <button type="button" class="btn-reset" data-reset>Clear</button>
      </div>
    </form>
    <input type="search" class="search" data-search="appointments"
placeholder="Search by patient, doctor, reason or status…" />
    <table id="appointments-table">
      <thead>
        <tr><th>ID</th><th>Patient</th><th>Doctor</th><th>Date</th>
<th>Reason</th><th>Status</th><th>Actions</th></tr>
      </thead>
      <tbody></tbody>
    </table>
  </section>

  <!-- REPORTS -->
  <section id="reports" class="panel">
    <h2>Reports</h2>

    <h3>Doctor workload &amp;amp; revenue</h3>
    <div class="report-controls">
      <label>Min. appointments
        <input type="number" id="workload-min" min="0" value="0" />
      </label>
    </div>
    <table id="workload-table">
      <thead>
        <tr><th>Doctor</th><th>Specialty</th><th>Fee (₹)</th><th>Total</th>
<th>Completed</th><th>Scheduled</th><th>Cancelled</th><th>Revenue (₹)</th></tr>
      </thead>
      <tbody></tbody>
    </table>

    <div class="grid-2">
      <div>
        <h3>Patients without appointments</h3>
        <table id="inactive-table">
          <thead><tr><th>Name</th><th>Age</th><th>Phone</th></tr></thead>
          <tbody></tbody>
        </table>
      </div>
```

```
      <div>
        <h3>Patients older than average age <span id="avg-age" class="muted">
</span></h3>
        <table id="above-avg-table">
          <thead><tr><th>Name</th><th>Age</th></tr></thead>
          <tbody></tbody>
        </table>
      </div>
    </div>
  </section>

  <div id="toast"></div>
  <script src="app.js"></script>
</body>
</html>
```

public/app.js

```
// Frontend CRUD logic. Talks to the Express API via fetch().

// ---- helpers -----------------------------------------------------------

async function api(path, options = {}) {
  const res = await fetch(`/api${path}`, {
    headers: { 'Content-Type': 'application/json' },
    ...options,
  });
  const data = await res.json().catch(() => ({}));
  if (!res.ok) throw new Error(data.error || `Request failed (${res.status})`);
  return data;
}

function toast(msg, isError = false) {
  const el = document.getElementById('toast');
  el.textContent = msg;
  el.className = isError ? 'show error' : 'show';
  setTimeout(() => (el.className = ''), 2200);
}

function esc(v) {
  if (v === null || v === undefined) return '';
  return String(v).replace(/[&<>"]/g, (c) =>
    ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[c]));
}

function formToObject(form) {
  const obj = {};
  new FormData(form).forEach((v, k) => { obj[k] = v === '' ? null : v; });
  return obj;
}

// ---- tab switching -----------------------------------------------------

document.querySelectorAll('.tab').forEach((btn) => {
  btn.addEventListener('click', () => {
    document.querySelectorAll('.tab').forEach((t) =>
t.classList.remove('active'));
    document.querySelectorAll('.panel').forEach((p) =>
p.classList.remove('active'));
    btn.classList.add('active');
    document.getElementById(btn.dataset.tab).classList.add('active');
    // Refresh views that depend on data changed in other tabs.
    if (btn.dataset.tab === 'dashboard') loadDashboard();
    if (btn.dataset.tab === 'reports') loadReports();
    if (btn.dataset.tab === 'appointments') fillAppointmentSelects();
  });
});

// Fill a read-only table body from rows; cells(row) returns the cell values.
function fillTable(sel, rows, cells, colspan) {
```

```
  const tbody = document.querySelector(`${sel} tbody`);
  tbody.innerHTML = rows.length
    ? rows.map((r) => `<tr>${cells(r).map((c) => `<td>${esc(c)}
</td>`).join('')}</tr>`).join('')
    : `<tr><td class="empty" colspan="${colspan}">No records.</td></tr>`;
}

// ---- generic entity controller -----------------------------------------

function makeEntity({ name, formId, tbodySel, columns, toForm }) {
  const form = document.getElementById(formId);
  const tbody = document.querySelector(tbodySel);
  const search = document.querySelector(`[data-search="${name}"]`);

  async function load() {
    try {
      const q = search.value.trim();
      const rows = await api(`/${name}${q ? `?q=${encodeURIComponent(q)}` :
''}`);
      render(rows);
      return rows;
    } catch (err) {
      toast(err.message, true);
      return [];
    }
  }

  function render(rows) {
    if (!rows.length) {
      tbody.innerHTML = `<tr><td class="empty" colspan="${columns.length +
2}">No matching records.</td></tr>`;
      return;
    }
    tbody.innerHTML = rows.map((r) => `
      <tr>
        <td>${r.id}</td>
        ${columns.map((c) => `<td>${esc(c.get ? c.get(r) : r[c.key])}
</td>`).join('')}
        <td>
          <button class="row-btn btn-edit" data-edit="${r.id}">Edit</button>
          <button class="row-btn btn-delete" data-del="${r.id}">Delete</button>
        </td>
      </tr>`).join('');

    tbody.querySelectorAll('[data-edit]').forEach((b) =>
      b.addEventListener('click', () => {
        const row = rows.find((x) => String(x.id) === b.dataset.edit);
        toForm(form, row);
        form.scrollIntoView({ behavior: 'smooth', block: 'center' });
      }));

    tbody.querySelectorAll('[data-del]').forEach((b) =>
      b.addEventListener('click', () => remove(b.dataset.del)));
  }
```

```
  async function remove(id) {
    if (!confirm('Delete this record?')) return;
    try {
      await api(`/${name}/${id}`, { method: 'DELETE' });
      toast('Deleted');
      await load();
    } catch (err) {
      toast(err.message, true);
    }
  }

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const body = formToObject(form);
    const id = body.id;
    delete body.id;
    try {
      if (id) {
        await api(`/${name}/${id}`, { method: 'PUT', body: JSON.stringify(body)
});
        toast('Updated');
      } else {
        await api(`/${name}`, { method: 'POST', body: JSON.stringify(body) });
        toast('Created');
      }
      form.reset();
      await load();
    } catch (err) {
      toast(err.message, true);
    }
  });

  form.querySelector('[data-reset]').addEventListener('click', () =>
form.reset());

  let searchTimer;
  search.addEventListener('input', () => {
    clearTimeout(searchTimer);
    searchTimer = setTimeout(load, 250);
  });

  return { load, form };
}

// ---- entity definitions ------------------------------------------------

const patients = makeEntity({
  name: 'patients',
  formId: 'patient-form',
  tbodySel: '#patients-table tbody',
  columns: [
    { key: 'name' }, { key: 'age' }, { key: 'gender' },
    { key: 'phone' }, { key: 'address' },
  ],
  toForm: (form, r) => {
```

```
    form.id.value = r.id;
    form.name.value = r.name ?? '';
    form.age.value = r.age ?? '';
    form.gender.value = r.gender ?? '';
    form.phone.value = r.phone ?? '';
    form.address.value = r.address ?? '';
  },
});

const doctors = makeEntity({
  name: 'doctors',
  formId: 'doctor-form',
  tbodySel: '#doctors-table tbody',
  columns: [{ key: 'name' }, { key: 'specialty' }, { key: 'phone' }, { key:
'consultation_fee' }],
  toForm: (form, r) => {
    form.id.value = r.id;
    form.name.value = r.name ?? '';
    form.specialty.value = r.specialty ?? '';
    form.phone.value = r.phone ?? '';
    form.consultation_fee.value = r.consultation_fee ?? '';
  },
});

const appointments = makeEntity({
  name: 'appointments',
  formId: 'appointment-form',
  tbodySel: '#appointments-table tbody',
  columns: [
    { key: 'patient_name' },
    { key: 'doctor_name' },
    { get: (r) => (r.appt_date ? r.appt_date.slice(0, 10) : '') },
    { key: 'reason' },
    { key: 'status' },
  ],
  toForm: (form, r) => {
    form.id.value = r.id;
    form.patient_id.value = r.patient_id ?? '';
    form.doctor_id.value = r.doctor_id ?? '';
    form.appt_date.value = r.appt_date ? r.appt_date.slice(0, 10) : '';
    form.reason.value = r.reason ?? '';
    form.status.value = r.status ?? 'scheduled';
  },
});

// Populate appointment dropdowns from patients + doctors.
async function fillAppointmentSelects() {
  try {
    const [ps, ds] = await Promise.all([api('/patients'), api('/doctors')]);
    const pSel = appointments.form.patient_id;
    const dSel = appointments.form.doctor_id;
    pSel.innerHTML = '<option value="">Patient *</option>' +
      ps.map((p) => `<option value="${p.id}">${esc(p.name)}
</option>`).join('');
    dSel.innerHTML = '<option value="">Doctor *</option>' +
```

```
      ds.map((d) => `<option value="${d.id}">${esc(d.name)}
</option>`).join('');
  } catch (err) {
    toast(err.message, true);
  }
}

// ---- dashboard ----------------------------------------------------------

async function loadDashboard() {
  try {
    const { totals, byStatus, upcoming } = await api('/reports/summary');
    document.getElementById('stats').innerHTML = [
      ['Patients', totals.patients],
      ['Doctors', totals.doctors],
      ['Appointments', totals.appointments],
      ['Upcoming', totals.upcoming],
    ].map(([label, value]) =>
      `<div class="stat"><div class="value">${value}</div><div
class="label">${label}</div></div>`).join('');
    fillTable('#status-table', byStatus, (r) => [r.status, r.count], 2);
    fillTable('#upcoming-table', upcoming,
      (r) => [r.appt_date, r.patient_name, r.doctor_name, r.reason], 4);
  } catch (err) {
    toast(err.message, true);
  }
}

// ---- reports ------------------------------------------------------------

const workloadMin = document.getElementById('workload-min');

async function loadReports() {
  try {
    const [workload, pats] = await Promise.all([
      api(`/reports/doctor-workload?min=${encodeURIComponent(workloadMin.value
|| 0)}`),
      api('/reports/patients'),
    ]);
    fillTable('#workload-table', workload, (r) => [
      r.name, r.specialty, r.consultation_fee, r.total,
      r.completed, r.scheduled, r.cancelled, r.revenue,
    ], 8);
    fillTable('#inactive-table', pats.inactive, (r) => [r.name, r.age,
r.phone], 3);
    fillTable('#above-avg-table', pats.aboveAvg, (r) => [r.name, r.age], 2);
    document.getElementById('avg-age').textContent =
      pats.aboveAvg.length ? `(avg ${pats.aboveAvg[0].avg_age})` : '';
  } catch (err) {
    toast(err.message, true);
  }
}

workloadMin.addEventListener('input', loadReports);
```

```
// ---- boot ------------------------------------------------------------

async function init() {
  await Promise.all([patients.load(), doctors.load(), appointments.load(),
loadDashboard()]);
  await fillAppointmentSelects();
}

init();
```

public/style.css

```
* { box-sizing: border-box; margin: 0; padding: 0; }

:root {
  --bg: #f4f6f9;
  --card: #ffffff;
  --primary: #2b6cb0;
  --primary-dark: #245a94;
  --danger: #e53e3e;
  --edit: #d69e2e;
  --border: #e2e8f0;
  --text: #1a202c;
  --muted: #718096;
}

body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-
serif;
  background: var(--bg);
  color: var(--text);
  line-height: 1.4;
  padding-bottom: 3rem;
}

header {
  background: var(--primary);
  color: #fff;
  padding: 1.4rem 1.5rem;
}
header h1 { font-size: 1.5rem; }
header p { opacity: 0.85; font-size: 0.85rem; margin-top: 0.2rem; }

.tabs {
  display: flex;
  gap: 0.25rem;
  background: var(--card);
  padding: 0 1rem;
  border-bottom: 1px solid var(--border);
  position: sticky;
  top: 0;
  z-index: 5;
}
.tab {
  border: none;
  background: none;
  padding: 0.9rem 1.2rem;
  font-size: 0.95rem;
  cursor: pointer;
  color: var(--muted);
  border-bottom: 3px solid transparent;
}
.tab.active {
  color: var(--primary);
```

```
  border-bottom-color: var(--primary);
  font-weight: 600;
}

.panel { display: none; padding: 1.5rem; max-width: 1000px; margin: 0 auto; }
.panel.active { display: block; }
.panel h2 { margin-bottom: 1rem; font-size: 1.25rem; }

.crud-form {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 1rem;
  display: flex;
  flex-wrap: wrap;
  gap: 0.6rem;
  margin-bottom: 1.5rem;
}
.crud-form input, .crud-form select {
  padding: 0.55rem 0.7rem;
  border: 1px solid var(--border);
  border-radius: 6px;
  font-size: 0.9rem;
  flex: 1 1 140px;
  min-width: 0;
}
.form-actions { display: flex; gap: 0.5rem; flex: 1 1 100%; }

button {
  cursor: pointer;
  border: none;
  border-radius: 6px;
  padding: 0.55rem 1rem;
  font-size: 0.88rem;
  font-weight: 500;
}
.btn-primary { background: var(--primary); color: #fff; }
.btn-primary:hover { background: var(--primary-dark); }
.btn-reset { background: #edf2f7; color: var(--text); }

table {
  width: 100%;
  border-collapse: collapse;
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 8px;
  overflow: hidden;
  font-size: 0.88rem;
}
th, td { text-align: left; padding: 0.7rem 0.8rem; border-bottom: 1px solid
var(--border); }
th { background: #f7fafc; color: var(--muted); font-weight: 600; text-
transform: uppercase; font-size: 0.72rem; letter-spacing: 0.03em; }
tr:last-child td { border-bottom: none; }
tr:hover td { background: #f7fafc; }
```

```
.row-btn {
  padding: 0.35rem 0.7rem;
  font-size: 0.78rem;
  margin-right: 0.35rem;
}
.btn-edit { background: var(--edit); color: #fff; }
.btn-delete { background: var(--danger); color: #fff; }

.empty { text-align: center; color: var(--muted); padding: 1.5rem; }

#toast {
  position: fixed;
  bottom: 1.2rem;
  left: 50%;
  transform: translateX(-50%) translateY(2rem);
  background: #2d3748;
  color: #fff;
  padding: 0.7rem 1.2rem;
  border-radius: 6px;
  font-size: 0.88rem;
  opacity: 0;
  transition: all 0.25s;
  pointer-events: none;
}
#toast.show { opacity: 1; transform: translateX(-50%) translateY(0); }
#toast.error { background: var(--danger); }

.search {
  width: 100%;
  padding: 0.55rem 0.7rem;
  border: 1px solid var(--border);
  border-radius: 6px;
  font-size: 0.9rem;
  margin-bottom: 0.8rem;
}

.panel h3 { font-size: 1rem; margin: 1.5rem 0 0.6rem; }
.muted { color: var(--muted); font-weight: 400; font-size: 0.85rem; }

.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
  gap: 0.8rem;
}
.stat {
  background: var(--card);
  border: 1px solid var(--border);
  border-radius: 8px;
  padding: 1rem;
}
.stat .value { font-size: 1.8rem; font-weight: 700; color: var(--primary); }
.stat .label { color: var(--muted); font-size: 0.8rem; text-transform:
uppercase; letter-spacing: 0.03em; }
```

```
.grid-2 {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 1.2rem;
}

.report-controls { margin-bottom: 0.6rem; font-size: 0.88rem; color: var(--
muted); }
.report-controls input {
  width: 5rem;
  margin-left: 0.4rem;
  padding: 0.35rem 0.5rem;
  border: 1px solid var(--border);
  border-radius: 6px;
}
```